

Thesis for the degree of master of science

Multi-Level Resource Management in 5G Network Slicing: Spectrum, Power and NOMA

Arouri Eya

Department of Information Convergence Engineering
The Graduate School
Pusan National University

August 2026

Multi-Level Resource Management in 5G Network Slicing: Spectrum, Power and NOMA

A Thesis submitted to the graduate school of
Pusan National University in partial fulfillment of
the requirements for the degree of Master of Science
under the direction of **Yoo Younghwan**

The thesis for the degree of Master of Science
by **Arouri Eya**
has been approved by the committee members.

Jun 29, 2026

Chair	Kim Jongdeok	_____
Member	Kim Wonseok	_____
Member	Yoo Younghwan	_____

Contents

1. Introduction	1
1.1. Research Background	1
1.2. Research Goals	2
1.3. Contributions	3
1.4. Thesis Overview	4
2. Background and Literature Review	5
2.1. Reinforcement Learning Fundamentals and Model-Free Baselines	5
2.1.1. Markov decision processes and the RL objective	5
2.1.2. Value-based deep RL: DQN	6
2.1.3. Policy-gradient methods: TRPO, PPO, A2C	6
2.1.4. Off-policy continuous control: DDPG, TD3, SAC, NAF	7
2.1.5. The sample-efficiency wall	7
2.2. Model-Based RL with Kernels: The KBRL Foundation	7
2.2.1. Why a kernel formulation suits the inter-slice problem	8
2.2.2. What [1] leaves out	9
2.3. NOMA	9
2.3.1. Learned NOMA in the recent literature	9
2.4. Hierarchical RL and Joint RB-and-Power Allocation in Sliced RANs	10
2.4.1. The hierarchical-RL framing	10
2.4.2. Joint RB-and-power allocation: classical optimisation	10
2.4.3. Joint RB-and-power allocation: deep RL	11
3. System Model and Problem Formulation	12
3.1. Network Slicing Model	12
3.1.1. Frame Structure and Resource Granularity	12
3.1.2. Slice Set and Service Classes	12
3.1.3. Service Level Agreement and Indicator Function	13
3.2. Two-Timescale Control Architecture	13
3.2.1. Decision stages and subframes	13

3.2.2. Inter-slice observation	14
3.2.3. Inter-Slice Action: Two-Dimensional Extension	15
3.2.4. Intra-Slice Observation and Action	16
3.2.5. Summary of The Contribution to the System Model	17
3.3. Inter-Slice Problem Formulation	17
3.3.1. Constrained Markov Decision Process	17
3.3.2. Per-Stage Decomposition	18
3.3.3. Solution Approach	19
3.4. Intra-Slice Problem Formulation	19
3.4.1. Per-TTI State and Action	19
3.4.2. Reward Structure	20
3.4.3. MDP Formulation	20
3.5. Coupling Between the Two Levels	20
3.5.1. The Indicator as a Functional of the Intra-Slice Trajectory	21
3.5.2. Transmission Flow Between Levels	21
4. Algorithm Design	22
4.1. Inter-Slice Algorithm: Two-Dimensional KBRL	22
4.1.1. Three-Module Architecture	22
4.1.2. Per Slice Classifier with the Projectron	23
4.1.3. H-Learner: Training with L-Shape Augmentation	24
4.1.4. E-Learner: Per-Margin Type-II Error Estimation	25
4.1.5. Controller: Two-Dimensional Decoupled Search	26
4.1.6. Overflow Projection	27
4.1.7. Cold Start Behavior	28
4.2. Intra-Slice Algorithm: NOMA Pairing with DQN Power Split	28
4.2.1. Per-TTI Pipeline	28
4.2.2. Classification and PRB-Budget Split (eMBB slices only)	29
4.2.3. SNR-Gap Pairing Heuristic	30
4.2.4. DQN Power-Split Agent	31
4.2.5. NOMA Reception and SIC Failure	32
4.2.6. OMA Fallback: Proportional-Fair Scheduling	33

4.3. One Decision Cycle	33
5. Numerical Evaluation	36
5.1. Simulation Environment	36
5.1.1. Traffic models	37
5.1.2. Channel models	37
5.1.3. Intra-slice scheduler	38
5.1.4. Scenarios	39
5.1.5. Algorithms Under Test	40
5.2. Hyperparameters of the Proposed System	41
5.3. Performance Evaluation	42
5.3.1. Scenario 1	42
5.3.2. Scenario 2	44
5.3.3. Scenario 3	45
5.3.4. Adjustments	47
5.3.5. Online classifier accuracy	48
5.4. Limitations and future Directions	48
6. Conclusion	51

List of Tables

Table 1: mMTC RAN Slice observed variables [1].	15
Table 2: eMBB RAN Slice observed variables [1].	15
Table 3: eMBB intra-slice observed variables.	18
Table 4: mMTC intra-slice observed variables.	18
Table 5: Simulator parameters.	37
Table 6: Scenarios used in this thesis.	40
Table 7: eMBB slice parameters.	40
Table 8: mMTC slice parameters.	41
Table 9: Inter-slice 2D KBRL controller.	42
Table 10: Intra-slice DQN.	42
Table 11: NOMA / channel-model parameters.	42
Table 12: Results for scenario 1: SLA violations, total RBs, and total power allocated.	44
Table 13: Results for scenario 2: SLA violations, total RBs, and total power allocated.	46
Table 14: Results for scenario 2: SLA violations, total RBs, and total power allocated.	46

List of Figures

Figure 1: Diagram of the control system [1].	23
Figure 2: L-Shape augmentation in the (r, p) plane.	26
Figure 3: Intra-slice per-TTI pipeline.	31
Figure 4: One decision cycle timing diagram.	35
Figure 5: Results for scenario 1.	44
Figure 6: Results for scenario 2.	45
Figure 7: Results for scenario 3.	47
Figure 8: Average number of adjustments per stage in each scenario.	48
Figure 9: Empirical prediction accuracy of the KBRL classifiers.	49

Multi-Level Resource Management in 5G Network Slicing: Spectrum, Power and NOMA

Arouri Eya

Department of Information Convergence Engineering
The Graduate School
Pusan National University

Abstract

Radio access network (RAN) slicing requires the slice controller to continuously partition resources such as physical resource blocks (PRBs) and transmit power so that each slice meets its service-level agreement (SLA). A recent kernel-based reinforcement-learning (KBRL) controller achieves cold-start safety on this problem and outperforms standard deep-reinforcement-learning baselines, but operates over a one-dimensional action space (PRBs only) and assumes a physics-free mMTC channel. This thesis extends KBRL along two axes, enlarged from PRB count to the joint (PRB count, per-PRB power) pair. An intra-slice non-orthogonal multiple-access (NOMA) controller is added for both eMBB and mMTC slices: UEs are paired by SINR gap and a deep Q-network selects each pair's power-split coefficient under successive interference cancellation. A 3GPP-compliant mMTC channel with QPSK demodulation threshold and SIC failure cascade replaces the physics-free mMTC layer of the original simulator. Across 30 seeds of three scenarios, the proposed 2D KBRL + NOMA controller reduces mean transmit power by 30–32 % and PRB occupation by 4–5 % relative to the 1D KBRL baseline at identical workloads while still maintaining the SLA requirements of each slice.

1. Introduction

1.1. Research Background

The fifth generation (5G) of mobile networks has consolidated network slicing as the standard mechanism for delivering heterogeneous services over a shared physical infrastructure [3]. A network slice is an end-to-end logical network deployed on the same radio-access, transport, and core resources as other slices, but operated under its own service-level agreement (SLA). 3GPP defines dedicated control-plane functions for slice selection, mobility, and session management [2], and comprehensive surveys document the architectural and softwarisation aspects of slicing in commercial deployments [4, 5, 6]. A single base station is now expected to serve, in the same time-frequency grid, slices dedicated to enhanced mobile broadband (eMBB) for high-rate streaming and download traffic, massive machine-type communications (mMTC) for very large populations of low-rate sensors, and ultra-reliable low-latency communications (URLLC) for sub-millisecond control traffic.

At the radio access network (RAN) level, each slice is allocated an exclusive subset of the physical-resource-block (PRB) grid and runs its own scheduler within that subset. The infrastructure provider's task is to decide, at every decision instant, how to partition the available PRBs and transmit-power budget across the active slices so that each SLA is met while no more resources than necessary are consumed. A static partition does not work: slice traffic loads vary on diurnal, weekly, and event-driven timescales, so a partition sized for the peak wastes resources off-peak and a partition sized for the average breaches SLAs during peaks. Dynamic partitioning is the standard answer, but it is a hard problem because the system state is only partially observed, the dynamics are stochastic, and the SLA outcome is revealed only at the end of each decision period as a delayed Boolean signal. These three properties — partial observation, stochastic dynamics, delayed scalar feedback — map directly onto the Markov decision process abstraction [9], which makes reinforcement learning (RL) the natural toolkit for the problem.

Two families of RL approaches dominate the recent literature: on-policy and

off-policy deep-RL methods, reviewed in §2.1. They share one decisive limitation in the present context, their exploration consumes orders of magnitude more environment interactions than an operating network can afford, and every exploratory step that breaches an SLA is a real, operator-visible violation. None of the published configurations of these algorithms satisfies a deployable cold-start safety profile.

The kernel-based model-based RL (KBRL) controller of [1] addresses this cold-start cost directly: it learns a per-slice classifier for SLA satisfaction rather than the full transition dynamics, and selects the smallest allocation predicted safe under a user-chosen reliability factor. The mechanism is detailed in §2.2. For the inter-slice resource-allocation problem under online cold-start safety, KBRL is the state of the art.

Two limitations of the published KBRL formulation, however, restrict its applicability to modern RAN deployments (§2.2.2): the action space is one-dimensional in PRBs only, leaving the inter-slice power axis outside the controller, and the simulator's mMTC layer is physics-free, advancing repetition counters deterministically without an SINR check or demodulation threshold. Together, these prevent any evaluation of power-domain control under realistic mMTC link behaviour.

A complementary line of work exploits the power-domain at the intra-slice level via non-orthogonal multiple access (NOMA) [20, 21], superposing paired-user signals on the same time-frequency resource at different power levels. The achievable region of NOMA strictly contains that of orthogonal access when the paired channel gains differ [22]. Learned NOMA controllers exist (§2.3.1) but none operates under a cold-start-safe inter-slice controller; all sit in the offline deep-RL regime.

1.2. Research Goals

This thesis asks whether the online-learning safety property of KBRL can be preserved when the inter-slice action is extended to jointly control PRBs and per-PRB transmit power, while also introducing a learned intra-slice NOMA

controller. The work is organised around four main questions:

- Can the cold-start safety property of kernel-MBRL be preserved when the inter-slice action is extended from PRBs to jointly managing PRBs and transmit power, together with an intra-slice NOMA controller?
- Does the proposed two-dimensional extension maintain the low online SLA-violation performance of the original 1D KBRL approach?
- Can the joint allocation of PRBs and per-PRB transmit power reduce overall transmit power and PRB usage compared to the 1D KBRL baseline under the same traffic conditions?
- Does the integration of a learned intra-slice NOMA mechanism — including SNR-gap user pairing, DQN-based power-split selection, and successive interference cancellation — improve performance compared to a conventional orthogonal multiple access scheme?

The evaluation presented in Chapter 5 addresses these questions using results obtained from thirty independent simulation runs across three representative network scenarios.

1.3. Contributions

This thesis provides four main contributions that address the limitations identified in [1]:

1. **A two-dimensional inter-slice KBRL controller.**

The original one-dimensional KBRL framework is extended from PRB-only allocation to a joint allocation of PRBs and per-PRB transmit power. The Projectron classifier is adapted to process two-dimensional inputs, while the sample augmentation mechanism is extended to preserve monotonicity across both resource dimensions. In addition, separate E-learners are used to estimate prediction reliability along the PRB and power axes.

2. **A learned intra-slice NOMA controller.**

Within each slice, users are paired according to their SNR differences,

and a deep Q-network (DQN) dynamically selects the NOMA power-splitting coefficient. Successive interference cancellation (SIC) is applied at the receiver, while unmatched users are served through a fallback orthogonal multiple access (OMA) scheme. The DQN operates online without requiring offline pre-training.

3. **A 3GPP-compliant mMTC channel model.**

The simplified mMTC model used in the original simulator is replaced with a more realistic channel model incorporating 3GPP TS 36.942 outdoor path loss, indoor penetration loss based on 3GPP TR 45.820 coverage classes, and a QPSK demodulation threshold. Unlike the original implementation, failed decoding attempts are explicitly modeled, allowing transmission failures and retransmissions to affect resource allocation decisions.

4. **A comprehensive empirical evaluation.**

The proposed framework is evaluated across three representative traffic scenarios and thirty independent simulation seeds per algorithm. Results show that the proposed 2D KBRL + NOMA framework reduces average transmit power by approximately 30–32% and PRB utilization by 4–5% compared to the original 1D KBRL baseline, while maintaining the SLA protection guarantees of the original approach.

1.4. Thesis Overview

Chapter 2 reviews the background related to 5G RAN slicing, reinforcement learning techniques, kernel-based online learning, and NOMA. Chapter 3 formulates the inter-slice and intra-slice resource allocation problems as constrained Markov decision processes. Chapter 4 presents the proposed two-dimensional KBRL framework together with the intra-slice NOMA mechanism and OMA fallback strategy. Chapter 5 describes the simulation setup, evaluation metrics, performance results across the considered scenarios and discusses limitations. Finally, Chapter 6 concludes the thesis.

2. Background and Literature Review

Three formulations of the inter-slice problem appear in the literature. The first two are mirror images of each other: one asks how few RBs can be used while keeping each slice's SLA-violation rate below its target [1], the other asks how well the per-slice QoS can be optimised given a fixed RB budget [8]. A third line of work [7] departs from this framing, jointly deciding sub-channels and per-RRH power to maximise tenant net profit (revenue from served users minus reservation cost), with SLA entering softly through a reward/penalty term. This thesis follows [1], since it aligns the interests of both the infrastructure provider — which benefits from releasing unused RBs — and the slice tenants, who need their SLA met.

The slicing literature has noted that radio-resource allocation in a sliced network can be factorised into two timescales [7]. At the inter-slice level, a slow controller decides how many resources each slice gets over a window of minutes. At the intra-slice level, a fast scheduler decides how to allocate those resources every few seconds.

Classical optimisation approaches [7] to inter-slice allocation — Lagrangian methods, queueing-theoretic approximations, model-predictive control — have been studied extensively. They are limited by their reliance on a closed-form model of the system: traffic arrivals, channel statistics, and queueing behaviour have to be specified up front, and the optimiser is only as good as that specification. An operating network drifts slowly in ways that no static model captures perfectly. Online learning is the standard answer.

2.1. Reinforcement Learning Fundamentals and Model-Free Baselines

2.1.1. Markov decision processes and the RL objective

A Markov decision process is a tuple (S, A, P, R, γ) where S is the state space,

A the action space, $P(s'|s,a)$ the transition kernel, $R(s,a)$ the reward function, and $\gamma \in (0,1)$ a discount factor. A policy $\pi: S \rightarrow A$ — possibly stochastic — induces a distribution over trajectories, and the RL objective is to find a policy that maximises the expected discounted return,

$$J(\pi) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

The Bellman equation for the state-value function $V^{\pi}(s) = E_{\pi} [\sum_t \gamma^t R(s_t, a_t) | s_0 = s]$ is

$$V^{\pi}(s) = \sum_a \pi(a|s) \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi}(s') \right].$$

Two families of methods solve this problem. Value-based methods learn V or its action-value variant $Q(s,a)$ and derive a policy from the value function. Policy-gradient methods parameterise the policy directly and optimise $J(\pi_{\theta})$ by gradient ascent.

In the RAN-slicing problem of [1] and of this thesis, S is the system observation defined in Chapter 3, A is the per-slice resource allocation, and R is a weighted combination of resource cost and SLA-violation penalty. The MDP is constrained — the SLA bound is best handled as a constraint rather than folded into the reward — and partially observed.

2.1.2. Value-based deep RL: DQN

DQN [10] replaces tabular Q-learning with a neural network mapping states to Q-values over discrete actions, stabilised by a replay buffer and a target network. It reached human-level Atari play but required 200 M frames per game. Successors — Double DQN [11], Dueling DQN, Rainbow — cut sample cost by 2–10 $\times$ without changing the regime.

2.1.3. Policy-gradient methods: TRPO, PPO, A2C

Policy-gradient estimators have high variance, and a series of refinements address this. TRPO [12] constrains each update to a KL ball around the current policy. PPO [13] replaces TRPO's hard KL constraint with a clipped surrogate

objective and is now a de facto default; PPO1 and PPO2 (the original and vectorised implementations in OpenAI Baselines) appear as separate baselines in [1]. A3C [14] runs multiple actors in parallel; this thesis uses its synchronous A2C variant from Stable-Baselines3, following the same convention as [1].

2.1.4. Off-policy continuous control: DDPG, TD3, SAC, NAF

Discrete-action Q-learning does not apply directly to continuous actions. DDPG [15] adapts actor-critic to continuous actions with DQN-style replay and target networks, but is unstable in practice. TD3 [16] mitigates Q-value overestimation with twin critics and delayed policy updates; SAC [17] adds an entropy term that stabilises training differently. NAF [18] parameterises the Q-function so that the maximiser over actions is closed-form, recovering Q-learning-style updates without an explicit actor. NAF is used in [1] because the per-slice RB-count action is treated as continuous after relaxation.

2.1.5. The sample-efficiency wall

These model-free baselines span the practical landscape of deep model-free RL at the time of [1]. They share one property that matters here above all others: they need many orders of magnitude more environment interactions to converge than an online setting can afford. For an agent trained offline against a simulator, that cost is invisible. For an agent that learns directly into SLA violations. An algorithm whose asymptotic performance is excellent but which would inflict a long tail of violations before reaching it is, for online RAN control, simply unusable.

This sample-efficiency wall is, for the purposes of this thesis, the decisive property of model-free RL. It is what makes a model-based approach — and specifically the kernel-based formulation of [1] — interesting.

2.2. Model-Based RL with Kernels: The KBRL Foundation

This section describes the kernel-based MBRL algorithm of [1], what it does,

why it works, and what it leaves out. The aim is to give the reader enough to understand the thesis's contribution; the formal extension and the implementation details belong to Chapter 4.

2.2.1. Why a kernel formulation suits the inter-slice problem

The SLA-conformance constraint partitions the action space into a safe region and an unsafe region; the optimal allocation is the cheapest action on the safe side. KBRL [1] exploits this directly: instead of learning the system's transition dynamics, it learns the per-slice SLA-violation indicator as a binary classifier and selects the smallest action the classifier labels safe. A kernel-based classifier represents the nonlinear safe/unsafe boundary, supports explicit online updates, and bounds its own memory footprint — three properties that matter when the agent must learn in thousands of stages rather than millions, with each exploratory step risking a real SLA violation. Specifically, [1] uses the Projectron [19], a bounded-memory variant of the kernel perceptron that projects each new misclassification onto the span of the existing support set, adding a new support vector only when projection error exceeds a threshold; the resulting support-set size is independent of the number of training samples. Built on this classifier, KBRL runs three modules per decision stage: an H-learner (one Projectron per slice, predicting SLA satisfaction for any candidate allocation), an E-learner (tracking each classifier's empirical error rate as a function of a security margin), and a controller that, for each slice, picks the smallest margin driving the E-learner's estimated error below the contracted bound $1-\delta$, finds the smallest action the H-learner labels safe, and outputs their sum; if the per-slice sum exceeds the capacity C , allocations are scaled down proportionally. Across multiple scenarios, [1] places KBRL against the seven model-free baselines of §2.2 and shows it sitting alone in the bottom-left corner of the resource-cost-vs-SLA-violation plane: lowest violation rate, among the most resource-efficient.

2.2.2. What [1] leaves out

The formulation of [1] controls only the number of RBs allocated to each slice. Per-RB transmit power is held fixed in the simulator at a calibration value chosen to place the resulting SINR distribution in the useful range of the LTE MCS table. This is a reasonable simplification for the inter-slice problem in isolation, but it leaves a degree of freedom — the power domain — entirely outside the controller. A second omission concerns mMTC link physics: the simulator advances per-sensor repetition counters deterministically as a function of allocated RBs, with no SINR check or demodulation threshold, so an mMTC sensor that should physically fail to decode (e.g., because per-PRB power has been reduced to the link budget's edge) is always counted as served. These two omissions motivate the 2D extension and the mMTC link-model addition of Chapter 4.

2.3. NOMA

2.3.1. Learned NOMA in the recent literature

[23] address downlink NOMA by deriving a closed-form power solution given subchannel assignment and learning the assignment via an attention-based deep-RL network. [24] extend this to multi-cell uplink, using MATD3 for power and DQN for cell association and subchannel assignment. [25] use matching theory for NOMA grouping in a two-slice cooperative-NOMA setting but treat transmit power as a fixed simulation parameter. All three treat NOMA as the only resource-allocation problem; none has an inter-slice controller above. The intra-slice controller in this thesis is a learned per-pair, per-instant power-split policy operating below the inter-slice level.

2.4. Hierarchical RL and Joint RB-and-Power Allocation in Sliced RANs

2.4.1. The hierarchical-RL framing

The two-timescale structure of the inter-slice and intra-slice problems fits

the options framework of [26], in which a high-level policy selects a temporally-extended sub-policy that runs for multiple steps before returning control upward. [27] survey the hierarchical-RL literature; modern instances include Option-Critic [28], a direct extension of the options framework, with related goal-based hierarchies such as FeUdal Networks [29]. The inter-slice controller is the high-level policy, the intra-slice scheduler is the option, and the SLA outcome plays the role of the termination signal.

2.4.2. Joint RB-and-power allocation: classical optimisation

The first line of work treats the problem as a constrained MINLP and applies mathematical programming. [30] formulate it for a multi-tenant multi-service network with eMBB, URLLC, and mMTC slices and solve it with a genetic algorithm under hard per-slice QoS constraints. [25] take a different route for cooperative-NOMA slicing with underlay D2D, decoupling the assignment into a two-stage swapping-based matching problem with transmit power fixed as a simulation parameter. Both formulations make feasibility against per-slice QoS explicit, but they assume closed-form channel models and are solved once for a static snapshot — adapting to time-varying conditions requires re-running the solver, which motivates the move to learning-based methods.

2.4.3. Hierarchical RL and Joint RB-and-power allocation using deep RL

Joint RB-and-power allocation has been tackled by deep-RL agents trained against simulators. [24] use DQN for cell association and subchannel assignment and TD3 for power in an uplink multi-cell NOMA system without slicing. [31] extend this to an H-NOMA uplink serving eMBB, mMTC, and URLLC: their CMADDQN handles subchannel assignment, with power obtained from an optimization-based routine (Dinkelbach plus Lagrangian sub-gradient, with closed-form per-user updates) for any given SA. This is the closest architectural precedent — joint subchannel-and-power control, hybrid optimization-plus-learning — but it is trained over many episodes against a

simulator, whereas the kernel-based formulation here learns from cold start on a live network within a few hundred decision stages. [32] take a different route: joint communication, computing, and caching allocation in a NOMA-based cell-free RAN with MEC, using a hierarchical MADDPG with a centralized slice-level agent (resource proportions) and per-slice user-level agents (NOMA clustering, RB allocation, offloading). This is the closest hierarchical precedent to the two-level structure here, but both levels are deep actor-critic learners trained offline for many episodes, and the per-slice action bundles clustering, RB allocation, and offloading together.

3. System Model and Problem Formulation

This work extends the inter-slice resource allocation problem formulated by [1] along two dimensions. First, the action space is generalized from the number of allocated resource blocks (RBs) per slice to the joint pair (RBs, power-per-RB), which permits the controller to trade spectrum against power. Second, a faster intra-slice control loop is introduced inside each slice. The two levels' loops (inter-slice and intra-slice) operate at different timescales — one decision per T_{step} TTIs at the inter-slice level, one decision per TTI at the intra-slice level — and are coupled through the SLA indicator, which evaluates each slice's aggregate performance over a full inter-slice period.

3.1. Network Slicing Model

3.1.1. Frame Structure and Resource Granularity

We consider a downlink transmission on an hexagonal cell similar to the scenarios described in [1], consisting of a base station providing access to multiple UEs belonging to K network slices, the radio interface is partitioned in time into subframes of duration $t_{\text{TTI}} = 1$ ms (one TTI), and in frequency into sub-carriers of spacing 15 kHz. The minimum time-frequency allocation unit is the resource block (RB), defined as one TTI by twelve sub-carriers; in each subframe a total of C RBs is available for downlink transmission. The base station is further constrained by a total transmit power budget P_{max} summed across all RBs and all slices in any given subframe.

Each RB is granted to at most one slice in any subframe — that is, RBs are not shared across slices — so that slice isolation is preserved at the radio level. The scheduler internal to a slice may then assign the slice's RBs to its UEs in accordance with the per-slice service type.

3.1.2. Slice Set and Service Classes

Let $K=1,2,\dots,K$ denote the set of active slices. Each slice $i \in K$ is associated

with a service class $\sigma_i \in \{\text{eMBB}, \text{mMTC}\}$ that determines (i) the traffic model of its UEs, (ii) the structure of its observation vector, and (iii) the form of its SLA. The set K is taken as fixed throughout this chapter; admission of new slices and termination of existing slices are out of scope and may be handled by an orthogonal admission-control mechanism such as those surveyed in [1]. accordance with the per-slice service type.

3.1.3. Service Level Agreement and Indicator Function

For each slice $i \in K$ and each decision stage n , define the binary SLA indicator

$$I^{(i)}(s_{n-1}^{(i)}, a_n^{(i)}) \in \{0, 1\},$$

which equals 1 if the SLA of slice i has been violated during the T_{step} TTIs spanned by stage n under the action $a_n^{(i)}$ taken at the start of the stage, and 0 otherwise. Following [1], the indicator condenses every key performance indicator listed in the slice's SLA into a single Boolean: it returns 0 only if all per-class QoS targets have been met during the stage.

Because $I^{(i)}$ depends on stochastic processes (UE arrivals and departures, traffic generation, fading), it is itself the realization of a random variable $I^{(i)}(S_{n-1}^{(i)}, A_n^{(i)})$. The control algorithm will reason about the probability that this random variable equals 1, not about $I^{(i)}$ directly. $I^{(i)}$ is class-specific: it aggregates throughput, PRB consumption, and queue length (per traffic type) for eMBB slices, and the average access delay for mMTC slices. The thresholds and the precise aggregation rule are deferred to Chapter 5.

3.2. Two-Timescale Control Architecture

3.2.1. Decision stages and subframes

Two distinct timescales coexist in the controlled system:

1. The inter-slice timescale, indexed by $n = 1, 2, \dots$, governs the allocation of RBs and power across slices. Consecutive inter-slice decisions are separated by T_{step} TTIs, hence one inter-slice decision is taken every T_{step} . The action

chosen at stage n remains in force throughout the subframes $t \in (n-1)T_{\text{step}} + 1, \dots, nT_{\text{step}}$.

2. The intra-slice timescale, indexed by $t=1,2,\dots$, governs the per-slice transmission decisions taken every TTI: UE classification, pairing, NOMA power splitting, and OMA scheduling. The intra-slice controller of slice i operates only on the resources allocated to slice i at the most recent inter-slice stage.

Table 1: mMTC RAN Slice observed variables [1].

variable	units
Simultaneous UE contexts	UEs
Remaining repetitions per UE	repetitions/UE
Average delay pe UE	seconds/UE

Table 2: eMBB RAN Slice observed variables [1].

variable	units
GBR incoming traffic	bits/s
GBR delivered traffic	bits/s
GBR average occupied resources	RBs/subframe
GBR average queue per user	bits/subframe
GBR average SINR per user	dB/subframe
non-GBR incoming traffic	bits/s
non-GBR delivered traffic	bits/s
non-GBR average occupied resources	RBs/subframe
non-GBR average queue per user	bits/subframe
non-GBR average SINR per user	dB/subframe

3.2.2. Inter-slice observation

At decision stage n , the inter-slice agent receives a per-slice observation vector $s_n^{(i)} \in \mathbb{R}^{d_i}, i \in \mathcal{K}$. where the dimension d_i depends on the service class σ_i . Each entry of $s_n^{(i)}$ is an aggregate of measurements collected during the T_{step} TTIs that ended at stage n . for instance, average throughput per traffic class, average buffer length, average SINR, and counts of active UEs. The full system observation is the concatenation

$$s_n = (s_n^{(1)}, s_n^{(2)}, \dots, s_n^{(K)}).$$

Following [1], we use the uppercase notation $\mathcal{S}_n^{(i)}$ to denote $s_n^{(i)}$ as a random variable and the lowercase $s_n^{(i)}$ to denote a particular realization. The vector s_n is a partial observation, not the full state of the system: the true state would have to encompass the positions, buffers, channel realizations, and protocol stacks of every UE in every slice and is therefore intractable to define or measure.

The inter-slice observation space is inherited verbatim from [1]; the 2D extension of this thesis modifies the action space but not the observation space. For eMBB, $d_{eMBB}=10$, split between guaranteed-bit-rate (GBR) and non-GBR traffic classes (Table 1). For mMTC, $d_{mMTC}=3$ (Table 2).

3.2.3. Inter-Slice Action: Two-Dimensional Extension

This is the first place the present work departs from [1]. In their formulation the action assigned to slice i at stage n is a scalar $a_n^{(i)} \in 0, 1, \dots, C$ giving the number of RBs allocated to slice i . We extend this action to the pair

$$a_n^{(i)} = (r_n^{(i)}, p_n^{(i)}) \in A^{(i)},$$

where $r_n^{(i)} \in \{0, 1, \dots, C\}$ is the number of RBs and $p_n^{(i)} \in P$ is the power allocated to each of those RBs, drawn from a finite grid

$$P = \{\Delta_p, 2\Delta_p, \dots, L\Delta_p\}.$$

The level count L and the step size Δ_p are configuration parameters of the architecture; the specific values used in our experiments are listed in Chapter 5. The total power consumed by slice i at stage n is therefore $P_n^{(i)} = r_n^{(i)} \cdot p_n^{(i)}$, and the joint action across all slices is $a_n = (a_n^{(1)}, \dots, a_n^{(K)})$.

An action a_n is admissible if it satisfies the joint capacity constraints

$$\sum_{i \in K} r_n^{(i)} \leq C \quad \text{and} \quad \sum_{i \in K} r_n^{(i)} \cdot p_n^{(i)} \leq P_{\max}. \quad 3.1$$

The set of admissible joint actions is denoted by A , and a control policy $\pi: s_n \mapsto a_n$ is admissible if it returns an element of A for every s_n . The set of admissible policies is denoted by Π .

The motivation for the two-dimensional extension is twofold. First, a slice can satisfy its SLA either with many RBs at low power per RB or with fewer RBs at high power per RB, and the optimal mix is in general state-dependent. Second, it makes the inter-slice action consistent with the underlying physical model of the air interface, in which the per-PRB SINR seen by each UE depends on both the assigned bandwidth and the transmit power, not on the bandwidth alone.

3.2.4. Intra-Slice Observation and Action

This is the second place where the present work departs from [1], in which intra-slice resource handling is left to a generic per-slice scheduler and is not modeled. We endow each slice with an explicit intra-slice controller that operates at TTI granularity.

At each TTI t within stage n , the controller of slice i observes the per-UE state vector

$$\xi_t^{(i)} = (\xi_t^{(i,1)}, \xi_t^{(i,2)}, \dots, \xi_t^{(i, U_t^{(i)})}),$$

where $U_t^{(i)}$ is the number of active UEs in slice i at TTI t and each $\xi_t^{(i,u)}$ is service-class specific (Tables 3 and 4): eMBB UEs carry byte queues and per-UE channel measurements, while mMTC sensors carry an integer count of remaining transmission repetitions and a scalar SINR. Based on $\xi_t^{(i)}$ and on the inter-slice allocation $(r_n^{(i)}, p_n^{(i)})$ in force at that TTI, the controller selects an intra-slice action

$$b_t^{(i)} = (\Pi_t^{(i)}, \alpha_t^{(i)}),$$

where $\Pi_t^{(i)}$ is a partition of the active UEs into NOMA pairs and OMA singletons, and $\alpha_t^{(i)} \in \mathcal{B} \subset [0.5, 1.0]$ is a finite vector of power-splitting ratios — one per NOMA pair, with each entry drawn from a discrete grid $\mathcal{B}$ subset $[0.5, 1.0]$. The partition $\Pi_t^{(i)}$ is produced by a fixed SNR-gap heuristic rather than learned; only the power-splitting vector $\alpha_t^{(i)}$ is learned, and it is selected per pair by a DQN whose parameters are shared across all pairs of slice i .

Table 3: eMBB intra-slice observed variables.

variable	units
Per-UE mean SINR	dB
Per-UE queue occupancy	bits
Number of active UEs	UEs
Inter-slice RB budget	PRBs
Inter-slice power-per-RB	W

Table 4: mMTC intra-slice observed variables.

variable	units
Per-UE SINR	dB
Per-UE remaining repetitions	repetitions
Number of active UEs	sensors
Inter-slice RB budget	PRBs
Inter-slice power-per-RB	W

3.2.5. Summary of The Contribution to the System Model

The system model differs from [1] in three respects, which together define the formal contribution of this chapter:

- (i) The inter-slice action space is generalized from $\{0, \dots, C\}$ to $\{0, \dots, C\} * P$, introducing power-per-RB as a control variable on equal footing with bandwidth.
- (ii) A second control loop is introduced at TTI granularity inside each slice, with its own observation $\xi_t^{(i)}$, its own action $b_t^{(i)}$, and its own decision policy.
- (iii) The SLA indicator $I^{(i)}$ is now a functional of the entire intra-slice trajectory $(b_t^{(i)})_{t=1}^{T_{\text{step}}}$, not only of the inter-slice action $a_n^{(i)}$ — a coupling made precise in Section 3.5.

3.3. Inter-Slice Problem Formulation

3.3.1. Constrained Markov Decision Process

The inter-slice control problem is the search, within the set $\mathcal{H}$ of admissible policies, for one that minimizes the long-run average resource consumption while keeping each slice's long-run SLA-violation rate below a specified bound

$\delta \in (0,1)$ Formally, we seek

$$\min_{\pi \in \Pi} \lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E}_{\pi} \left[\sum_{n=1}^N \sum_{i \in K} (w_r R_n^{(i)} + w_p R_n^{(i)} P_n^{(i)}) \right] \quad 3.2$$

subject to

$$\lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E}_{\pi} \left[\sum_{n=1}^N I^{(i)}(S_{n-1}^{(i)}, A_n^{(i)}) \right] \leq \delta, \quad \forall i \in K, \quad 3.3$$

and to the per-stage admissibility constraints (3.1) embedded in Π . Here $R_n^{(i)}$ and $P_n^{(i)} = R_n^{(i)} p_n^{(i)}$ are the random RB and power consumption of slice i at stage n , and $w_r, w_p \geq 0$ are weights that scalarize the bi-objective spectrum/power cost. This formulation specializes to the CMDP of [1, eq. (1)] in the limit $w_p = 0$ and $P = p_{\text{ref}}$ (a single power level), which recovers the original RB-only problem. we set $w_r = 1/C$ and $w_p = 0$, so that the objective reduces to the per-stage average RB consumption normalized to $[0,1]$, the same cost minimized in [1], Power does not appear in the cost; it enters the formulation only through the joint admissibility constraint $\sum_i r_n^{(i)} p_n^{(i)} \leq P_{\text{max}}$, which is enforced by a separate projection step rather than by the search itself. The per-slice controller of Section 4.1.5 minimizes RBs first and then trims power-per-RB at the chosen RB count, using the smallest power needed to keep the slice classifier-safe; this is a minimum-resource step, not a cost-minimization step under $w_p = 0$. The added power dimension is therefore what extends [1], the action space grows, while the cost is preserved.

3.3.2. Per-Stage Decomposition

The CMDP (3.2)–(3.3) cannot be solved directly: the system dynamics are unknown, the state is only partially observed through s_n , and the joint action space grows combinatorially in K . Following [1], we approximate the long-run problem by a sequence of one-step lookahead problems, one per decision stage, using a learned per-slice classifier $h_n^{(i)}$ that predicts the SLA outcome and a learned error-probability estimator $\widehat{e}_n^{(i)}$ that quantifies the classifier's reliability.

At stage n , the controller solves

$$\min_{a_n \in A} \sum_{i \in K} (w_r r_n^{(i)} + w_p r_n^{(i)} p_n^{(i)}) \quad 3.4$$

subject to

$$\widehat{e}_n^{(i)}(h_n^{(i)}, s_{n-1}^{(i)}, a_n^{(i)}) \leq \delta, \quad \forall i \in K, \quad 3.5$$

and to (3.1). The per-stage problem (3.4)–(3.5) decomposes across slices up to the joint capacity constraint, and is the operational problem actually faced by the controller.

The construction of $h_n^{(i)}$ and $\widehat{e}_n^{(i)}$, the search procedure that returns $a_n^{(i)}$ from these objects, the augmentation strategy that exploits monotonicity in the action, and the projection used to enforce (3.1) are algorithmic mechanisms; they are described in Chapter 4.

3.3.3. Solution Approach

The approach taken in this work is model-based reinforcement learning with kernel classifiers (KBRL), introduced for the one-dimensional action by [1] and extended here to the two-dimensional action of Section 3.2.3. The model consists of K independent online kernel classifiers, one per slice, each predicting whether a candidate action will satisfy its slice's SLA. The reliability of each classifier is monitored by a paired self-assessment estimator that drives an adaptive safety margin. The detailed mechanics of the classifier, the self-assessment update, the two-dimensional resource search, and the L-shape sample augmentation are deferred to Chapter 4.

3.4. Intra-Slice Problem Formulation

3.4.1. Per-TTI State and Action

Within a single inter-slice stage n , the resources granted to slice i are fixed at $(r_n^{(i)}, p_n^{(i)})$, and the intra-slice controller must decide, at every TTI t in the stage, how to use them to serve the active UEs of the slice. We formulate this

as a per-slice MDP whose state is the intra-slice observation $\xi_t^{(i)}$ defined in Section 3.2.4 and whose action is the power-splitting vector $\alpha_t^{(i)}$ alone, one discrete ratio per NOMA pair, each drawn from the grid B. The partition $\Pi_t^{(i)}$ is not part of the MDP action: it is produced deterministically from $\xi_t^{(i)}$ by the SNR-gap heuristic of Section 4.2.3 and is therefore folded into the state's preprocessing.

3.4.2. Reward Structure

The intra-slice reward at TTI t in slice i is class-dependent. For an eMBB slice, the reward reflects the per-TTI throughput delivered to the paired UEs, normalized by the maximum throughput attainable on the same resources; for an mMTC slice, the reward counts the number of UEs in the pair whose transmissions succeed at TTI t . In both cases, the reward is local to the pair and is summed across pairs to produce the slice-level reward. The exact functional forms are stated in Chapter 4 alongside the DQN training procedure.

3.4.3. MDP Formulation

For each slice $i \in \mathcal{K}$, define the per-slice MDP

$$\mathcal{M}^{(i)} = (\Xi^{(i)}, \mathcal{B}^{(i)}, P^{(i)}, R^{(i)}, \gamma),$$

where $\Xi^{(i)}$ is the space of intra-slice observations, $\mathcal{B}^{(i)}$ is the action space of $\alpha_t^{(i)}$ — i.e., $\mathcal{B}^{|\Pi_t^{(i)}|}$, with one discrete ratio per pair formed by the SNR-gap heuristic — $P^{(i)}$ is the unknown transition kernel (which absorbs the deterministic pairing rule), $R^{(i)}$ is the per-TTI reward of Section 3.4.2, and $\gamma \in (0,1)$ is a discount factor. The intra-slice controller's task is to learn a policy $\mu^{(i)} : \Xi^{(i)} \rightarrow \mathcal{B}^{(i)}$ that maximizes the expected discounted return over each inter-slice stage. Because the state and action spaces are large, this work approximates $\mu^{(i)}$ with a deep Q-network trained online, as detailed in Chapter 4.

3.5. Coupling Between the Two Levels

3.5.1. The Indicator as a Functional of the Intra-Slice Trajectory

Once the intra-slice loop is part of the system, the SLA indicator $I^{(i)}$ used in Section 3.2.3 can no longer be regarded as a function of the inter-slice action $a_n^{(i)}$ alone. It is, more precisely, a functional of the entire intra-slice trajectory executed during stage n :

$$I^{(i)} = I^{(i)}(s_{n-1}^{(i)}, a_n^{(i)}, (\xi_t^{(i)}, b_t^{(i)})_{t=1}^{T_{\text{step}}}),$$

where $I^{(i)}$ aggregates the per-TTI outcomes (delivered bits, delays, successful transmissions) over the stage and applies the slice's SLA rule. The inter-slice agent does not observe this trajectory directly; it observes only the resulting indicator and the next-stage observation $s_n^{(i)}$. From the inter-slice agent's point of view the intra-slice loop is therefore a black box whose effect is folded into the conditional distribution $\Pr(I^{(i)}=1 | s_{n-1}^{(i)}, a_n^{(i)})$ that the classifier $h_n^{(i)}$ is trying to learn.

3.5.2. Transmission Flow Between Levels

The coupling between the two levels is asymmetric. The inter-slice level transmits $(r_n^{(i)}, p_n^{(i)})$ downward to slice i 's intra-slice controller, which treats this pair as a fixed resource budget for the next T_{step} TTIs. The intra-slice level transmits, upward and only once per stage, two summaries: the next-stage observation $s_n^{(i)}$ — itself an aggregate of the per-TTI quantities — and the SLA indicator $I^{(i)}$. The two controllers do not exchange any other information, in particular, the inter-slice agent has no access to the per-TTI rewards used to train the intra-slice DQN, and the intra-slice agent has no knowledge of the classifier or its security margins. This separation is what allows the two loops to be designed and analyzed independently in Chapter 4.

4. Algorithm Design

Chapter 3 cast the resource-allocation problem as two coupled optimization problems: an inter-slice constrained Markov decision process (CMDP) reduced to a per-stage classifier-driven program, and a per-slice intra-slice MDP operating at TTI granularity. This chapter specifies the algorithms that solve them.

4.1. Inter-Slice Algorithm; Two-Dimensional KBRL

4.1.1. Three-Module Architecture

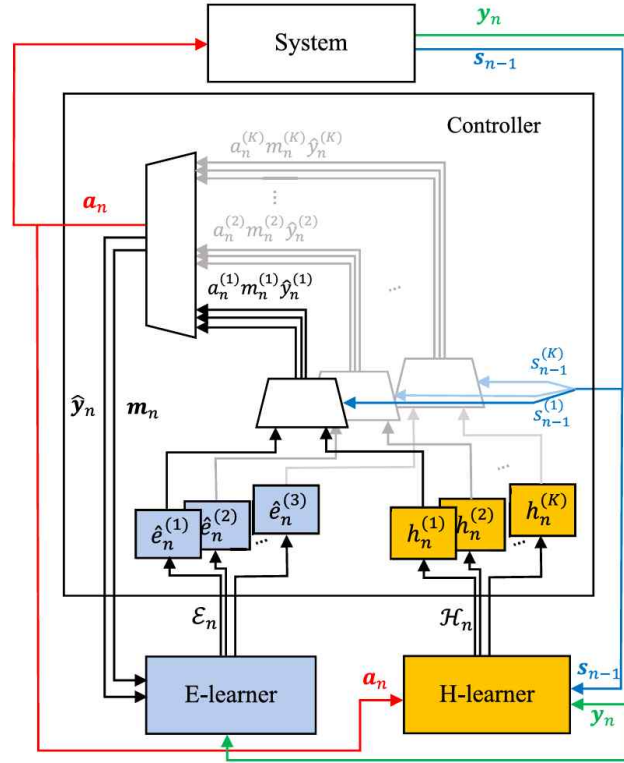


Figure 1: Diagram of the control system [1].

Following [1], the inter-slice agent is organized into three modules that act once per decision stage n in the order shown in Figure 1:

Let $H_n = \{h_n^{(1)}, \dots, h_n^{(K)}\}$ denote the set of per-slice classifiers and $E_n = \{\hat{e}_n^{(1)}, \dots, \hat{e}_n^{(K)}\}$ the corresponding error estimators at stage n . The agent is parameterized by the resource budgets $(C, P_{\max})$, the maximum per-slice action $a_{\max}^{(i)}$, and the reliability

factor $\bar{\delta}=1-\delta$ defined in Section 3.3.1. Algorithm 1 summarizes one stage of the inter-slice agent; the modules it calls are detailed in the subsequent subsections.

ALGORITHM 1: KBRL stage

```

1: Inputs:  $C, P_{\max}, \bar{\delta}, a_{\max}^{(i)}$  for  $i \in K$ 
2: Initialize  $H_1, E_1$ , observe  $s_0$ 
3: for  $n=1,2,\dots$  do
4:    $(a_n, m_n, \hat{y}_n) \leftarrow \text{Controller}(H_n, E_n, s_{n-1})$ 
5:   Apply  $a_n$ , observe  $y_n$  and  $s_n$ 
6:    $H_{n+1} \leftarrow \text{H-learner}(H_n, y_n, s_{n-1}, a_n)$ 
7:    $E_{n+1} \leftarrow \text{E-learner}(E_n, \hat{y}_n, y_n, m_n)$ 
8: end for

```

4.1.2. Per Slice Classifier with the Projectron

For each slice $i \in K$, the classifier $h_n^{(i)}$ is a kernel expansion

$$h_n^{(i)}(x) = \sum_{x_j \in X_n^{(i)}} \alpha_j^{(i)} \kappa(x_j, x), \quad \hat{y} = \text{sgn}(h_n^{(i)}(x)), \quad 4.1$$

where $X_n^{(i)}$ is the support set, $\alpha_j^{(i)}$ are the kernel coefficients, and κ is the Gaussian kernel $\kappa(x, x') = \exp(-\gamma_\kappa |x - x'|^2)$ with bandwidth γ_κ set in Chapter 5. The output convention is the one used in [1] when phrased as a classifier: $\hat{y} = +1$ predicts SLA satisfied (safe), $\hat{y} = -1$ predicts SLA violated (unsafe).

The classifier input is the concatenation of the per-slice observation and the candidate action,

$$x = (s_{n-1}^{(i)}, \tilde{r}, \tilde{p}), \quad \tilde{r} = r/C, \quad \tilde{p} = p/p_{\max}, \quad 4.2$$

where the action coordinates are normalized so that all input dimensions lie in $[0,1]$. The input dimension is therefore $d_i + 2$. This is the first concrete extension of [1], in which the action contributes a single normalized coordinate $\tilde{a}$ rather than the pair $(\tilde{r}, \tilde{p})$.

Each $h_n^{(i)}$ is updated with the Projectron online algorithm of [1]. When a new training pair (x, y) produces a prediction error $y \cdot h_n^{(i)}(x) \leq 0$, the Projectron first attempts to express the would-be update $h_n^{(i)} + y\kappa(x, \cdot)$ as a linear combination

of the existing support vectors. The projection coefficients are $d^* = K^{-1}k_x$ where K is the kernel matrix on $X_n^{(i)}$ and k_x is the vector of kernel values between x and the elements of $X_n^{(i)}$. If the residual $\delta = \kappa(x, x) - d^*k_x$ is below a threshold η , the new sample is approximated by existing support vectors and only the coefficients are updated. Otherwise, x is added to $X_n^{(i)}$ and the inverse kernel matrix is expanded incrementally. This bounded-budget mechanism is what keeps the per-stage cost of the classifier sub-quadratic in the number of decisions taken so far.

4.1.3. H-Learner; Training with L-Shape Augmentation

The H-learner receives, for each slice i , the observation-action pair $(s_{n-1}^{(i)}, a_n^{(i)})$ that was applied at stage n and the realized label $y_n^{(i)}$, and updates $h_n^{(i)} \rightarrow h_{n+1}^{(i)}$. To accelerate learning, it does not update the classifier with the observed pair only: it generates additional training pairs that share the same observation $s_{n-1}^{(i)}$ but differ in the action coordinate, exploiting the following monotonicity assumption inherited from [1] and extended to two dimensions.

Assumption 4.1 (Monotonicity in resources).

For every slice $i \in K$ and every observation $s_{n-1}^{(i)}$, increasing either r at fixed p or p at fixed r cannot turn an SLA-satisfying action into an SLA-violating one. Symmetrically, decreasing either coordinate cannot turn an SLA-violating action into an SLA-satisfying one.

Under Assumption 4.1, the set of safe actions in the (r, p) plane is upward-closed in the componentwise order, and the set of unsafe actions is downward-closed. The augmentation rule used by the H-learner reflects this geometry directly:

- if $y_n^{(i)} = +1$ (safe), every action with $r' \geq r_n^{(i)}$ and $p' \geq p_n^{(i)}$ is also safe. The H-learner generates synthetic training pairs along the L-shaped frontier $(r', p_n^{(i)}) : r' \geq r_n^{(i)} \cup \{(r_n^{(i)}, p') : p' \geq p_n^{(i)}\}$ and updates the classifier with label $+1$ for each.

– if $y_n^{(i)} = -1$ (unsafe), every action with $r' \leq r_n^{(i)}$ and $p' \leq p_n^{(i)}$ is also unsafe. The synthetic pairs are generated on the mirrored L-shape (Figure 2) with label -1 . This is the second extension of [1]: their augmentation generates synthetic samples along the single action axis a , whereas in two dimensions the augmentation must respect monotonicity in both coordinates simultaneously, hence the L-shape.

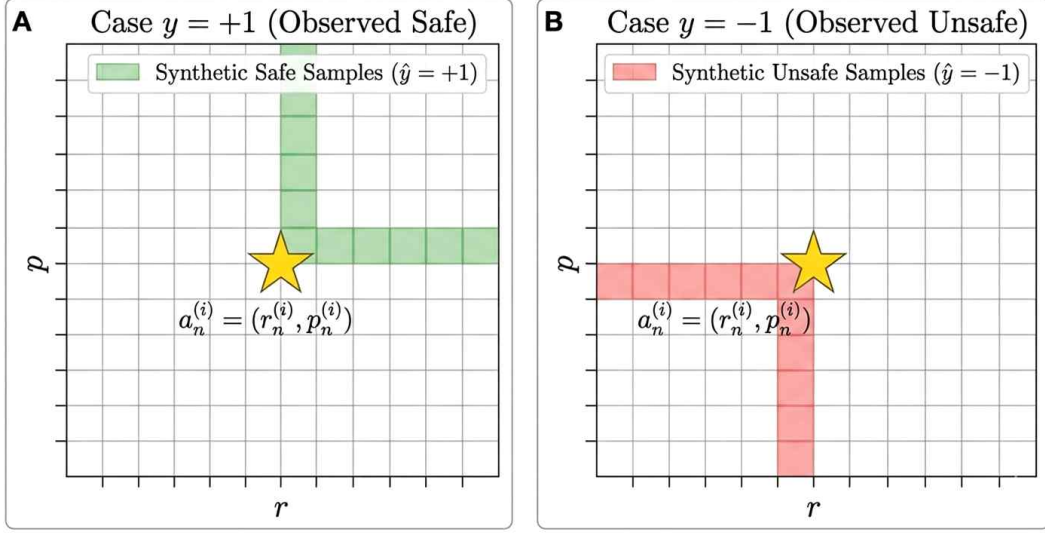


Figure 2: L-Shape augmentation in the (r, p) plane.

ALGORITHM 2: H-learner stage

Inputs: H_n, s_{n-1}, a_n, y_n

```

1: for  $i = 1, \dots, K$  do
2:   if  $y_n^{(i)} = +1$  then
3:     for  $(r, p) \in$  upper-right L of  $a_n^{(i)}$  do
4:       Projectron update of  $h_n^{(i)}$  with input  $(s_{n-1}^{(i)}, r/C, p/p_{\max})$  and label  $+1$ 
5:     end for
6:   else
7:     for  $(r, p) \in$  lower-left L of  $a_n^{(i)}$  do
8:       Projectron update with label  $-1$ 
9:     end for
10:  end if
11: end for

```

4.1.4. E-Learner: Per-Margin Type-II Error Estimation

The classifier output is binary, but the controller needs a calibrated sense of how confident the classifier is at any given action. Rather than a single confidence number, the E-learner maintains a discretized error profile indexed by a security margin, the gap, measured along each action dimension, between the candidate action and the boundary of the predicted-safe region.

For slice i , define the per-dimension margins

$$m_r = r - \min\{r' : h_n^{(i)}(s_{n-1}^{(i)}, r', p) = +1\} \quad m_p = p - \min\{p' : h_n^{(i)}(s_{n-1}^{(i)}, r, p') = +1\}.$$

For each slice i and each dimension $\zeta \in \{r, p\}$, the E-learner maintains an array $\hat{e}_{n,\zeta}^{(i)} : 0, 1, \dots, M_\zeta$ that estimates, for each margin level m , the type-II error probability of the classifier, that is, the probability that an action predicted safe with margin m is in fact SLA-violating. This is the third extension of [1], whose E-learner maintains a single error profile because there is only one action dimension.

The update rule is the exponential moving average of [1, eq. (9)] applied separately on each dimension:

$$\hat{e}_{n+1,\zeta}^{(i)}(m) = (1 - \beta)\hat{e}_{n,\zeta}^{(i)}(m) + \beta 1_{y_n^{(i)} = -1} \quad 4.3$$

applied only when $\hat{y}_n^{(i)} = +1$ (no type-II error is possible when the controller predicts unsafe), and only for margins consistent with the realized outcome under Assumption 4.1: a confirmed safe outcome decays $\hat{e}$ at margins $m \geq m_n^{(i)}$, while a violation inflates $\hat{e}$ at margins $m \leq m_n^{(i)}$. The learning rate β is set in Chapter 5.

4.1.5. Controller: Two-Dimensional Decoupled Search

The controller solves the per-stage problem (3.4)–(3.5) of Chapter 3, which after decomposition reduces, for each slice i , to the search for the cheapest action (r, p) such that $h_n^{(i)}(s_{n-1}^{(i)}, r, p) = +1$ and $\hat{e}_{n,\zeta}^{(i)}(m_\zeta(r, p)) \leq \delta$ for $\zeta \in \{r, p\}$

Solving this jointly in two dimensions is unnecessarily expensive: the classifier already encodes the safe region, and Assumption 4.1 makes that region a staircase in the (r, p) plane. The controller therefore performs a decoupled two-phase search.

Phase 1: RB search at maximum power. Fix $p = p_{\max}$ and binary search the smallest RB count $r^\dagger$ such that $h_n^{(i)}(s_{n-1}^{(i)}, r^\dagger, p_{\max}) = +1$. This is exactly the one-dimensional search of [1] specialized to the upper boundary of the power grid.

Phase 2 — Power search at $r^\dagger$. Fix $r = r^\dagger$ and binary search the smallest power level $p^\dagger$ such that $h_n^{(i)}(s_{n-1}^{(i)}, r^\dagger, p^\dagger) = +1$. Together, $(r^\dagger, p^\dagger)$ is the corner of the safe staircase that lies above and to the right of the slice's preferred operating point.

If Phase 1 returns no safe RB count, the controller falls back to the maximum admissible action $(C, p_{\max})$ for that slice. The two security margins are then read off the E-learner:

$$m_{n,r}^{(i)} = \min\{m : \hat{e}_{n,r}^{(i)}(m) \leq \delta\}, \quad m_{n,p}^{(i)} = \min\{m : \hat{e}_{n,p}^{(i)}(m) \leq \delta\}$$

and the action returned to the system is

$$a_n^{(i)} = (\min(C, r^\dagger + m_{n,r}^{(i)}), \min(p_{\max}, p^\dagger + m_{n,p}^{(i)})). \quad 4.4$$

The decoupled search is justified, rather than just convenient, by the upward-closed structure of the safe region under Assumption 4.1: minimizing r first at maximum p identifies the leftmost column of the safe staircase, after which minimizing p at that column descends to the bottom-left corner of that column.

4.1.6. Overflow Projection

The per-slice actions returned by (4.4) need not jointly satisfy the admissibility constraint (3.1). When they do not, the controller projects onto the admissible set by proportional scaling:

$$\bar{r}_n^{(i)} = \frac{C \cdot r_n^{(i)}}{\sum_{j \in K} r_n^{(j)}} \text{ if } \sum_j r_n^{(j)} > C,$$

and analogously for power, with the resulting power-per-RB re-quantized to the grid P . This projection is identical in form to [1, Algorithm 5, lines 16–22] but is applied independently to the two budgets, and the new value of the security margins is corrected accordingly so that the E-learner update at the next stage

uses the projected margin rather than the pre-projection one. The latter correction is essential: without it, the E-learner would attribute the SLA outcome to an action the system never executed.

4.1.7. Cold Start Behavior

At stage $n=1$, the support sets $X_1^{(i)}$ are empty and the classifier output is undefined. During this initial transient the controller falls back to a fixed conservative action — typically $(r_{\text{init}}^{(i)}, p_{\text{max}})$ where $r_{\text{init}}^{(i)}$ is slice-type-specific — and the security margins are initialized at the midpoint of their target accuracy band. As augmented samples accumulate, the classifier transitions from this fallback regime to data-driven operation; in the experiments of Chapter 6 this transition completes within the first few hundred stages.

4.2. Intra-Slice Algorithm: NOMA Pairing with DQN Power Split

The intra-slice algorithm is novel to this work. Each slice i runs an independent instance of the algorithm described below at every TTI t , operating on the resources $(r_n^{(i)}, p_n^{(i)})$ allocated by the inter-slice controller at the most recent decision stage.

4.2.1. Per-TTI Pipeline

At every TTI t within stage n , the intra-slice controller of slice i executes a per-TTI pipeline whose structure depends on the slice's traffic type.

eMBB slices: carry byte-queue traffic and execute the full five-step pipeline shown in Figure 3:

1. UE classification into backlogged and light sets, based on a comparison between each UE's queue and its OMA fair-share capacity (Section 4.2.2).
2. PRB-budget split between a NOMA pool and an OMA pool (Section 4.2.2).
3. Pairing of backlogged UEs by an SNR-gap heuristic (Section 4.2.3).
4. Selection of a power-splitting ratio α for each pair by a deep Q-network

(Section 4.2.4).

5. Reception, with SIC at the strong UE (Section 4.3.5), and proportional-fair scheduling of the OMA pool (Section 4.3.6).

mMTC slices: carry fixed-size repetition-coded packets, with no byte-queue to compare against an OMA fair-share, the per device demand is uniform and atomic. Steps 1 and 2 therefore have no operational meaning for mMTC and are omitted. The mMTC controller executes a three-step variant (Figure 3):

1. Pairing of all active sensors at TTI t by SNR, capped at the slice's PRB budget (Section 4.3.3).
2. DQN selection of α per pair (Section 4.3.4).
3. Reception with SIC at the strong sensor; sensors that do not fit into pairs are transmitted single-tone and decoded against a QPSK SNR threshold (Section 4.3.5).

Pairing and DQN-based α selection are functionally identical across the two classes; only their inputs differ (per-PRB SNR vectors and byte queues for eMBB, scalar SNR and remaining-repetition counts for mMTC). The reception step shares SIC at the strong device in both classes, but the handling of OMA/unpaired devices differs: eMBB uses proportional-fair scheduling with MCS adaptation, while mMTC uses single-user-per-PRB transmission decoded against a hard QPSK SNR threshold.

4.2.2. Classification and PRB-Budget Split (eMBB slices only)

Let $U_t^{(i)}$ be the set of active UEs in slice i at TTI t , and let $\overline{\eta}_u$ be the spectral efficiency that UE u would obtain at its own mean SNR over the slice's PRBs. The OMA fair-share capacity of UE u , the number of bits UE u would receive if the slice's PRBs were divided equally among its UEs is

$$C_u^{\text{OMA}} = \overline{\eta}_u \cdot N_{\text{sym}} \cdot [r_n^{(i)} / |U_t^{(i)}|],$$

where N_{sym} is the number of data symbols per PRB. UE u is classified as light if its queue $q_u \leq C_u^{\text{OMA}}$ and as backlogged otherwise. Light UEs cannot benefit from NOMA, they have nothing to push beyond their fair share, and are routed to the

OMA pool. The slice's $r_n^{(i)}$ PRBs are split between the two pools by serving all light UEs first at their nominal demand and assigning the remainder to the NOMA pool. Let N_{light} be the (capped) number of PRBs required to clear the light queues, the NOMA pool receives $N_{NOMA} = r_n^{(i)} - N_{light}$ PRBs.

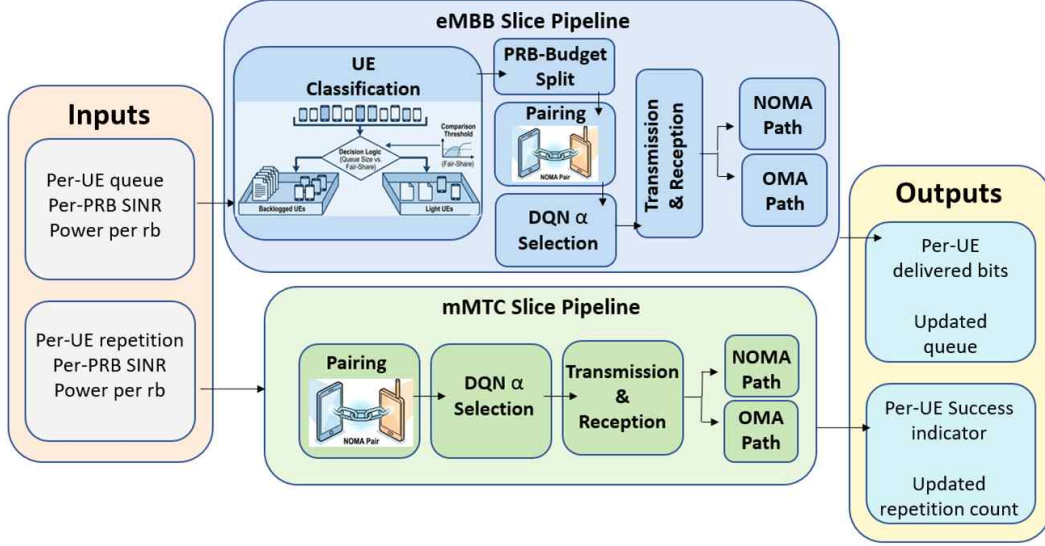


Figure 3: Intra-slice per-TTI pipeline.

4.2.3. SNR-Gap Pairing Heuristic

This step applies identically to both eMBB and mMTC slices and differs only in the set of pairing-eligible devices. Pairing in NOMA benefits from large channel disparity between the two UEs of a pair: the strong UE has more residual capacity to spare to SIC overhead, and the weak UE benefits more from the dedicated power share. Following the standard heuristic in the NOMA scheduling literature, the eligible devices are sorted by mean SNR in descending order and paired strongest-with-weakest:

$$\text{Pair}_k = (u_{(k)}, u_{(N_b - k + 1)}), \quad k = 1, \dots, \lfloor N_b/2 \rfloor,$$

where N_b is the number of eligible devices and $u_{(k)}$ is the device with the k -th highest SNR. The number of pairs actually formed is capped by the number of NOMA PRBs available, devices that are not paired fall back to the OMA pool (eMBB) or to a transmission decoded against a hard QPSK SINR threshold

(mMTC).

The pairing is a fixed heuristic, not part of the learned policy. The DQN of the next subsection sees the pairing only through its inputs (the two UEs' SINRs and queues, plus the inter-slice power budget) and does not influence which UEs are paired.

4.2.4. DQN Power-Split Agent

For each NOMA pair, a deep Q-network selects the power-splitting ratio that allocates fraction $\alpha \in \mathcal{B}$ of the per-PRB power to the weak UE and fraction $1 - \alpha$ to the strong UE, with the discrete grid

$$\mathcal{B} = \{0.50, 0.55, 0.60, \dots, 0.95, 1.00\}.$$

The choice $\alpha \geq 0.5$ encodes the standard NOMA convention that the weaker channel receives at least half the power, which is also the condition under which SIC at the strong UE is well-posed.

The DQN's input for a given pair (u_s, u_w) at TTI t is the five-dimensional vector

$$\xi_t^{(i,k)} = (\overline{\text{SNR}}_{u_s,t}, \overline{\text{SNR}}_{u_w,t}, q_{u_s,t}, q_{u_w,t}, p_n^{(i)}),$$

where $\overline{\text{SNR}}$ is the device's mean per-PRB SNR estimate for eMBB UEs and the device's scalar nominal SNR for mMTC UEs (which use narrowband flat fading), $q_{u,t}$ is the device's work-remaining state — the byte-queue occupancy for eMBB and the remaining transmission-repetition count for mMTC, and $p_n^{(i)}$ is the inter-slice power per RB in force at TTI t . The two service classes share the DQN's input vector structure but use class-specific normalisation constants for these inputs (Chapter 5). The DQN is trained with the standard ingredients of [15]: ϵ -greedy exploration with ϵ decayed from 1.0 to 0.05 an experience replay buffer, and a target network with soft updates. Specific hyperparameters are listed in Chapter 5.

The reward delivered to the DQN at TTI t for pair k is class-dependent:

- eMBB: The reward is the realized fraction of the pair's NOMA-feasible bits that were actually decoded,

$$R_{t,k}^{\text{eMBB}} = \frac{b_{u_s,t} 1\{u_s \text{ succeeds}\} + b_{u_w,t} 1\{u_w \text{ succeeds}\}}{\max(b_{u_s,t} + b_{u_w,t}, 1)} \in [0, 1],$$

where $b_{u,t}$ is the attempted bits at the NOMA-feasible MCS — i.e., the bits the link-adaptation table assigns to UE u when fed the per-PRB-averaged NOMA SNR (post-SIC for the strong UE, with-interference for the weak UE) on the pair's allocated PRBs. The denominator is therefore the bits the NOMA waveform would deliver under perfect reception, not a hypothetical OMA baseline; the reward measures how much of what NOMA promised the link actually delivered.

- mMTC: The reward counts successful transmissions in the pair,

$$R_{t,k}^{\text{mMTC}} = 1\{u_s \text{ succeeds}\} + 1\{u_w \text{ succeeds}\} \in \{0, 1, 2\}$$

One DQN agent is instantiated per slice; pairs within the same slice share the agent's parameters but are evaluated independently.

4.2.5. NOMA Reception and SIC Failure

For a pair sharing N_p PRBs, the base station transmits a superposed signal on each shared PRB n ,

$$x(n) = \sqrt{\alpha p_n^{(i)}} x_{w(n)} + \sqrt{(1-\alpha) p_n^{(i)}} x_{s(n)},$$

where x_w and x_s are the unit-power data symbols of the weak and strong UE respectively. Decoding follows the standard NOMA recipe:

- The weak UE treats the strong UE's signal as interference, with per-PRB SNR

$$\gamma_w(n) = \frac{\alpha |h_w(n)|^2}{(1-\alpha) |h_w(n)|^2 + N_0},$$

where $h_w(n)$ is the weak UE's channel coefficient on PRB n and N_0 is the noise power per PRB.

- The strong UE first decodes the weak UE's signal at SNR

$$\gamma_{w@s}(n) = \frac{\alpha |h_s(n)|^2}{(1-\alpha) |h_s(n)|^2 + N_0},$$

subtracts it, and decodes its own signal at the post-SIC SNR

$$\gamma_s(n) = \frac{(1 - \alpha)|h_s(n)|^2}{N_0}.$$

Because $|h_s|^2 > |h_w|^2$ by the pairing rule, the strong UE sees the weak UE's signal at higher SNR than the weak UE itself does, which is why SIC is feasible at the strong UE but not at the weak UE.

SIC is not always successful, however, and the algorithm models this explicitly. For eMBB pairs, the success probability of the SIC step is read from the link-level reception model at the weak UE's MCS evaluated at $\gamma_{w@s}$, with a Bernoulli draw determining the realization; when SIC fails, the strong UE's reception is also declared a failure. For mMTC pairs, the simpler hard-threshold rule of the link-level model is applied: SIC fails if γ_w falls below the QPSK demodulation threshold. The link-level reception model itself, the fitted reception probability sigmoids, the mutual-information-based effective-SNR mapping, and the MCS table, is described in Chapter 5.

4.2.6. OMA Fallback: Proportional-Fair Scheduling

Light UEs and unpaired backlogged UEs are served on the OMA pool of N_{light} PRBs by a proportional-fair scheduler. PRBs are assigned in groups of two, for each group, the scheduler selects the UE that maximizes

$$u_t^\star = \arg \max_u \frac{\overline{\eta_u} \cdot N_{sym} \cdot 1 q_{u,t}}{T_{u,t}} > 0$$

where $T_{u,t}$ is the exponential moving average of UE u 's past throughput,

$$T_{u,t} = (1 - \frac{1}{W})T_{u,t-1} + \frac{1}{W} \frac{b_{u,t-1}}{\tau_{TTI}},$$

with window length W . The scheduler is identical to the standard proportional-fair rule used as a baseline in [1] for intra-slice allocation; what differs is only that here it operates on a subset of the slice's PRBs, the rest having been routed to the NOMA pool.

4.3. One Decision Cycle

To anchor the foregoing in a single timeline, this section walks through one complete inter-slice decision cycle (Figure 4).

Step 1: Inter-slice observation. At the boundary stage n , the inter-slice agent reads the system observation s_{n-1} , itself constructed as the aggregate of per-TTI quantities measured during stage $n-1$.

Step 2 : Inter-slice action. The controller of Section 4.1.5 runs the two-phase decoupled search for each slice, the security margins of Section 4.1.4 are added, the joint admissibility (3.1) is enforced by the projection of Section 4.2.6, and the resulting $a_n = ((r_n^{(1)}, p_n^{(1)}), \dots, (r_n^{(K)}, p_n^{(K)}))$ is dispatched to the slices.

Step 3 : Intra-slice execution for T_{step} TTIs. Each slice runs the per-TTI pipeline of Section 4.2.1. The DQN learns from per-TTI rewards at the same timescale; the inter-slice agent does not see these rewards. UE arrivals, departures, and traffic generation evolve stochastically; the per-PRB SNR of each UE evolves with the fading trace.

Step 4 : SLA evaluation, At the end of stage n , each slice's aggregate KPIs are computed and compared against the SLA rule of Section 3.1.3, producing the indicator $y_n^{(i)} = I^{(i)} \in \{0, 1\}$. The next-stage observation $s_n^{(i)}$ is also computed from the per-TTI records.

Step 5 : Inter-slice learning. The H-learner of Section 4.1.3 updates each $h_n^{(i)}$ using the triple $(s_{n-1}^{(i)}, a_n^{(i)}, y_n^{(i)})$ together with its L-shape augmented expansion, and the E-learner of Section 4.1.4 updates the per-margin error estimators. The state used in this update is the pre-action observation $s_{n-1}^{(i)}$, not $s_n^{(i)}$, because the classifier predicts the SLA outcome that would result from taking an action in a given state.

Step 6 : Loop. Stage $n+1$ begins, using the freshly updated model (H_{n+1}, E_{n+1}) and the new observation s_n .

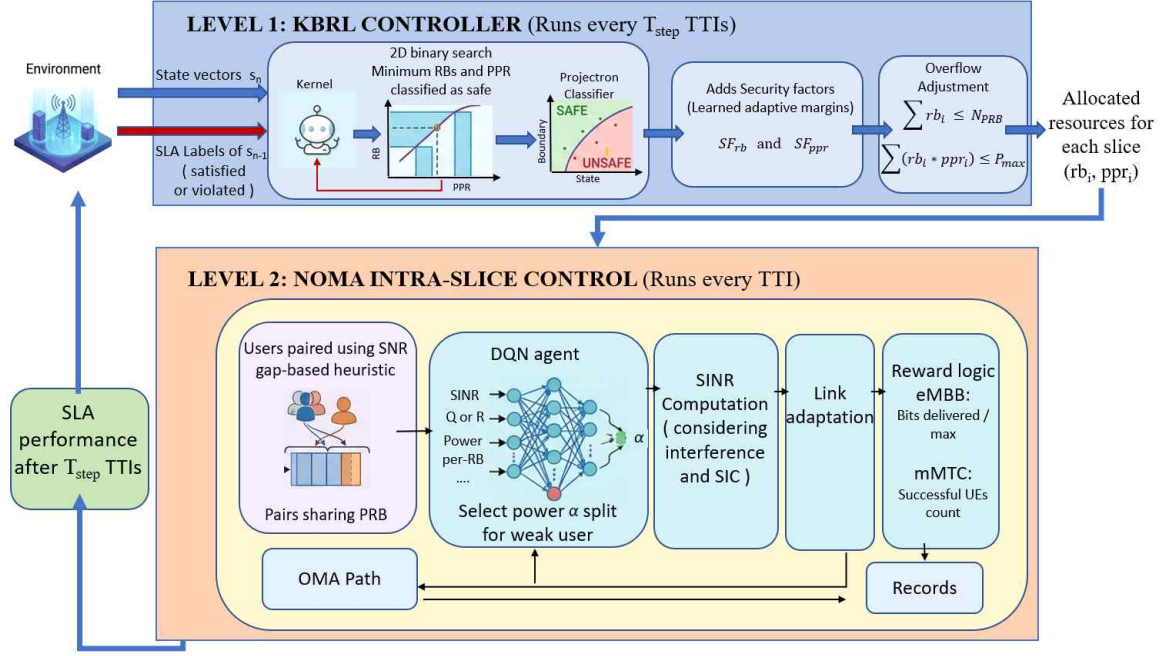


Figure 4: One decision cycle timing diagram.

5. Numerical Evaluation

This chapter quantifies the contribution of the two-level architecture of Chapter 4 against the one-dimensional KBRL baseline of [1] on the same simulator and the same scenarios as the original work.

5.1. Simulation Environment

The simulator is the Python environment of [1], extended to expose the per-RB transmit power as a controlled axis and to model the mMTC link at the physical layer. Each slice is devoted to one service type — eMBB or mMTC — and is characterised by its traffic source, its SLA and, in the case of eMBB, an intra-slice scheduler that distributes the slice's allocated resources among its UEs. The slice-allocation decision is updated every $T_{\text{step}} = 50$ radio subframes of 1 ms each, the elapsed time between two consecutive inter-slice decision stages is therefore 50 ms. The DQN of Chapter 4 operates at the per-TTI (1 ms) timescale on the resources fixed at the most recent stage boundary. Base parameters of the simulator are summarised in Table 5.

Table 5: Simulator parameters.

Parameter	Value
TTI length	1 ms
TTIs per decision stage T_{step}	50
Decision-stage length	50 ms
Number of stages per run T	50,400
Propagation model	<i>macro cell urban 2 GHz</i> <i>Section 4.5.1, TS 36.942 [34]</i>
Fading model	<i>Pedestrian A, Typical Urban,</i> <i>Vehicular A (Annex B.2, TS 36.104 [35])</i>
Reference transmit power P_{ref}	30 dBm (= 1 per PRB)
Antenna gain (max)	15 dBi
BS antenna pattern	<i>Section 4.2.1, TS 36.942 [34]</i>
Cell radius R_{max}	2 km
Noise figure	9 dB
Interference + noise floor	− 110 dBm/PRB
Outdoor log-normal shadow σ_{LSF}	10 dB
Number of independent seeds per run	30

5.1.1. Traffic models

Each eMBB slice generates two classes of traffic flows (inherited from [1]): GBR (guaranteed-bit-rate) flows simulated as constant-bit-rate (CBR) sources, and non-GBR flows simulated as variable-bit-rate (VBR) sources. Inter-arrival times and session durations are exponentially distributed; the parameters are listed in Table 7. For each new UE arrival the simulator draws a Poisson session start, a fading trace and a random position in the hexagonal cell. Each mMTC slice instantiates 1000 stationary sensors, Each sensor is characterised by a transmission period T_{arr} (the inter-arrival time between its packet generations, drawn once at sensor creation) and an initial repetition count n_{rep} (Table 8). The repetition count tracks how many transmission opportunities a sensor still has before it is removed from the queue; in the 1D baseline [1] it is decremented unconditionally each time the sensor obtains a PRB, whereas in the 2D + NOMA setting it is set to zero on a successful decode and decremented on a failed one.

5.1.2. Channel models

The simulator distinguishes two channel models, one per service class. The eMBB model is inherited verbatim from [1], the mMTC model is introduced by this thesis to make the inter-slice power axis actionable at the link level. The eMBB SNR per UE per PRB is the sum of a large-scale component (TR 36.942 [33], ‘macro_cell’ path-loss model evaluated at the UE’s randomly drawn position inside the hexagonal cell) and a small-scale component drawn from a pre-computed fading traces. The trace files correspond to usual fading/mobility models (pedestrian A, Typical Urban, Vehicular A, as defined in Annex B.2 of TS 36.104 [34]). At insertion time, each new UE is randomly assigned one of the three available traces, together with a random starting index and a random walk direction through that trace. The MCS table and the associated mutual-information-based effective-SNR mapping are also inherited from [1].

The 1D simulator of [1] does not model the mMTC link at the physical layer: its mMTC slot procedure advances repetition counters as a function of the

allocated RBs only, with no SNR or transmit-power dependence. To make the per-RB transmit power introduced in Chapter 4 actionable at the mMTC link level, this thesis adds a per-sensor SNR model whose generation rule is $SNR_u = P_{tx, dBm} - L_{macro}(d_u) - L_{LSF} - L_{MCL} - N_0$, where $L_{macro}(d_u)$ is the outdoor TS 36.942 `macro\_cell` [34] path loss at the sensor's sampled distance, L_{LSF} is an independent log-normal shadow draw [33]. N_0 is the thermal-noise floor and L_{MCL} is a fixed indoor-penetration offset chosen, The offsets $\{0, 10, 20\}$ dB are re-purposed from the coupling-loss extensions of the 3GPP NB-IoT Coverage Enhancement classes CE-0, CE-1, CE-2 of NB-IoT [35] which span the 20 dB coverage-extension target above the 144 dB GSM baseline established by 3GPP TR 45.820 [36] section 4.1.1. A custom 3 dB bin is added for finer granularity. We adopt this discrete-offset abstraction in preference to the composite COST-231 penetration model of TR 45.820 Annex D.1 because the focus is on RAN-slice scheduling under per-RB power control rather than per-link coverage statistics: fixed offsets per scenario isolate the effect of penetration depth on slice-level KPIs while keeping the channel deterministic across Monte Carlo runs. Small-scale fading is omitted at the system level (stationary sensors, narrowband 180 kHz signal, fading effects absorbed into receiver sensitivity). The scenarios evaluated in this thesis use mild (3 db) by default.

5.1.3. Intra-slice scheduler

Within each slice, an intra-slice scheduler distributes the slice's allocated Resources among its active UEs at the per-TTI timescale. The scheduler depends on the service type.

The eMBB intra-slice scheduling: The eMBB scheduler is the proportional-fair (PF) scheduler inherited verbatim from [1]. At every TTI it iterates over the slice's PRBs in groups of two and assigns each group to the UE that maximises $\eta_{u(t)} \cdot 1\{q_{u(t)} > 0\} / T_{u(t)}$, where $\eta_{u(t)}$ is the UE's achievable spectral efficiency at its current estimated SNR, $q_{u(t)}$ is its queue occupancy and $T_{u(t)}$ is a sliding-window exponential moving average of its past throughput (window length $W=50$ TTIs). The chosen MCS for each UE is selected from the LTE MCS table targeting a

block error rate (BLER) below 0.1 at the UE's effective SNR. In the 2D + NOMA setting (Chapter 4), the PF scheduler is restricted to the OMA pool, the NOMA pool is served instead by the DQN-driven pairing of Section 4.3.

The mMTC intra-slice scheduling: The mMTC layer-1 module does not invoke a PF scheduler. Active sensors are served in FIFO order until the slice's PRB budget is exhausted: the first active sensors transmit each TTI. In the 1D baseline [1], each transmission unconditionally decrements the sensor's repetition counter, in the 2D + NOMA setting, the post-NOMA SNR is checked against the QPSK demodulation threshold (6 dB), with a successful decode setting the counter to zero and a failed decode decrementing it.

5.1.4. Scenarios

Three scenarios from [1] are used, indexed 1–3. Each scenario fixes the slice mix and the cell's total PRB and power budget, the simulator instances one cell per scenario (Table 6).

Table 6: Scenarios used in this thesis.

Scenario	$C(\text{PRBs})$	$P_{BS}(W)$	K_{eMBB}	K_{mMTC}
1	200	200	5	0
2	150	150	3	2
3	100	100	1	4

Each scenario reuses the eMBB and mMTC slice configurations of [1] (traffic models for eMBB, SLA bounds, observation variables) without modification (Tables 7 and 8).

Table 7: eMBB slice parameters.

Parameter	Value
CBR arrival rate λ^{CBR}	$2/60 \text{ s}^{-1}$
CBR mean session duration	30 s
CBR target bit rate	500 kbps
VBR arrival rate λ^{VBR}	$5/60 \text{ s}^{-1}$
VBR mean session duration	30 s
VBR packet size	1000 bits
VBR burst size	500
VBR burst rate	1

SLA (CBR throughput)	$\geq 10\text{Mbps}$
SLA (CBR PRBs occupied)	$\leq 20\text{PRBs/subframe}$
SLA (CBR queue)	$\leq 10^5\text{bits/subframe}$
SLA (VBR throughput)	$\geq 15\text{Mbps}$
SLA (VBR PRBs occupied)	$\leq 30\text{PRBs/subframe}$
SLA (VBR queue)	$\leq 1.5 \times 10^5\text{bits/subframe}$

Table 8: mMTC slice parameters.

Parameter	Value
Total mMTC devices in slice	1000
Repetition set	{2, 4, 8, 16, 32, 64, 128}
Period set (TTIs)	{1000, 5000, 10,000, 15,000, 20,000, 25,000, 50,000, 100,000}
SLA bound on average delay	300 <i>TTIs</i>
Coverage scenario	“mild” (3 dB indoor penetration)

5.1.5. Algorithms Under Test

The comparison is between the proposed system and the family of controllers already evaluated by [1] on the same simulator. The proposed system is the only one with access to the per-RB power axis and the intra-slice NOMA pipeline; the others operate on the 1D RB-only action space of [1] with transmit power held at $P_{ref} = 30$ dBm.

- KBRL-1D: the kernel-MBRL controller of [1], reproduced from the original code. The primary head-to-head comparison.
- NAF: Normalised Advantage Functions [18], deep-RL baseline reported by [1].
- PPO1, PPO2: Proximal Policy Optimization [13][37], the original and the vectorised implementations used by [1].
- TRPO: Trust Region Policy Optimization [12] from [1].
- A2C: synchronous Advantage Actor-Critic [14] from [1].
- TD3: Twin Delayed DDPG [16] from [1].
- SAC: Soft Actor-Critic [17] from [1].

All seven 1D baselines have already been benchmarked by [1] on the same scenarios, simulator, SLA bounds, and 30-seed protocol that this thesis re-uses, with the same observation space and the same per-stage decision granularity.

The point of the comparison is to show that the 2D extension plus intra-slice NOMA outperforms KBRL-1D itself, on per-slice SLA fulfilment and on resource efficiency, while preserving the cold-start safety property that distinguishes KBRL from the deep-RL family.

5.2. Hyperparameters of the Proposed System

All hyperparameters of the inter-slice 2D KBRL controller, the intra-slice DQN, and the channel model are collected in Tables 9–11.

Table 9: Inter-slice 2D KBRL controller.

Parameter	Value
Power-per-RB ceiling $p_{\max}$	1.0 W/PRB
Power-grid resolution L	11 levels
Initial (r, p) per slice	<i>slice-type-specific</i>
Accuracy target (δ)	0.03 or 0.01
Accuracy band (low end)	0.97 or 0.99
E-learner β parameter	0.01
Projectron η parameter	0.1
Projectron Kernel	<i>Gaussian with $\sigma = 1$</i>

Table 10: Intra-slice DQN.

Parameter	Value
Input dimension	5
Output dimension	11
Hidden layers	2 layers $\times$ 64 units
Learning rate	10^{-3}
Discount factor γ	0.95
Exploration ϵ (start/end)	1.0/0.05
ϵ decay horizon	10,000 steps
Replay buffer size	50,000
Mini-batch size	32
Train period	every 4 steps
Target update period	every 100 steps
Soft update factor τ	0.01

Table 11: NOMA / channel-model parameters.

Parameter	Value
Power-split grid B	(0.50, 0.55, ..., 1.00) (11 levels)
SIC model (eMBB)	Bernoulli draw at $p_{\text{sic}} = \text{BLER}^{-1}(\text{MCS}_{\text{w}}, \gamma_{\text{w@s}})$
SIC model (mMTC)	Hard threshold; SIC fails iff $(\gamma_{\text{w}} < \gamma_{\text{QPSK}})$

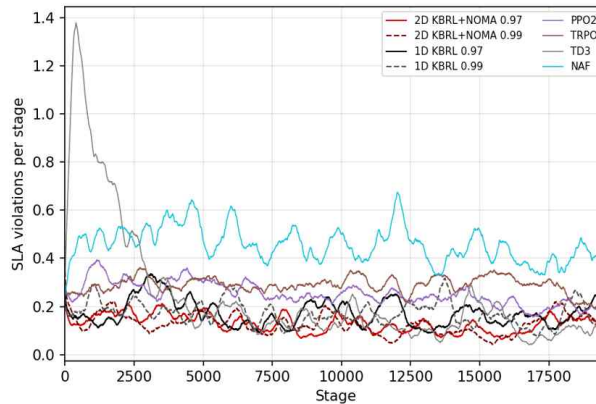
QPSK SNR threshold γ_{QPSK}	-6 dB
Indoor penetration scenarios	$\{0,3,10,20\}$ dB (normal / mild / extended / extreme)
Scenario used in this evaluation	“mild” ($+3\text{ dB}$)

5.3. Performance Evaluation

This section follows the structure of the online-performance evaluation in [1]. In each of the three scenarios we executed 30 independent simulation runs per algorithm. Each run comprises a learning phase of 40 000 decision stages, during which the algorithms learn from scratch without any prior knowledge of the system. The proposed KBRL controller is evaluated for two reliability factors $\bar{\delta} \in \{0.97, 0.99\}$, in both the 1D (RB-only) and 2D (RB + power) variants. [1] includes a second phase of 10 000 stages where the policy is frozen and the algorithms are compared once trained, and has already established that 1D KBRL dominates A2C, PPO1, PPO2, TRPO, SAC, TD3 and NAF, in both the learning and the inference phases, across all three scenarios. Re-litigating that comparison is not the point of this thesis: the open question is whether the 2D + NOMA controller proposed here actually improves on the 1D KBRL baseline.

5.3.1. Scenario 1

Figure 5 and Table 12 reports the first 20 000 stages of the learning phase for scenario 1. From top to bottom the three panels show SLA violations per stage, total RBs allocated, and total power allocated per stage.



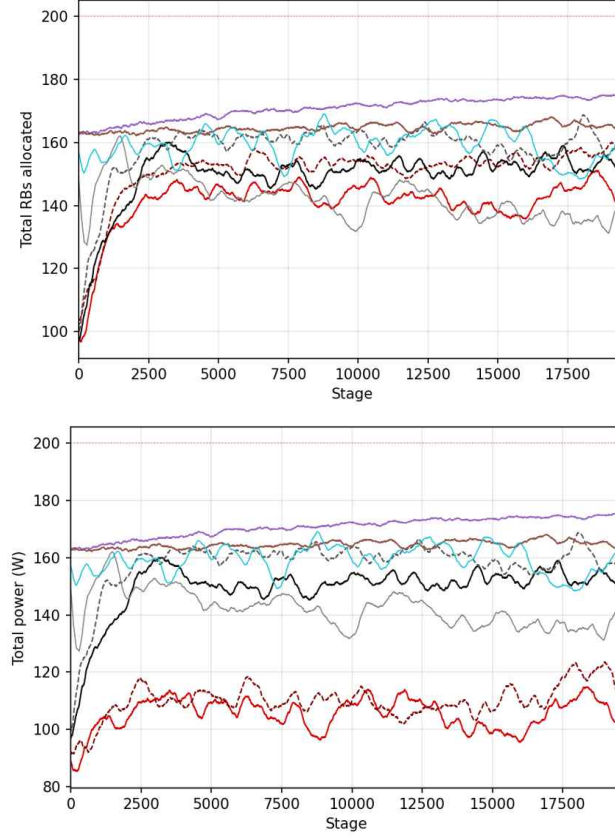


Figure 5: Results for scenario 1.

Table 12: Results for scenario 1: SLA violations, total RBs, and total power allocated.

Algorithm	Viol/stage	Cum. viol.	Mean RB	Mean Power (W)
KBRL $\bar{\delta}=0.97$ — 1D	0.1703	6 813	150.06	150.06
KBRL $\bar{\delta}=0.99$ — 1D	0.1709	6 836	158.29	158.29
KBRL+NOMA $\bar{\delta}=0.97$ — 2D	0.1168	4 670	142.26	105.80
KBRL+NOMA $\bar{\delta}=0.99$ — 2D	0.1094	4 375	151.33	109.73
PPO2	0.2240	8 958	173.65	173.65
TRPO	0.2542	10 169	165.26	165.26
TD3	0.1763	7 051	137.38	137.38
NAF	0.4400	16 900	158.86	158.86

Compared with the corresponding 1D KBRL baselines, the proposed 2D + NOMA controller reduces the mean violation rate by approximately 31.4% at $\bar{\delta}=0.97$ and 36.0% at $\bar{\delta}=0.99$. The mean RB allocation is reduced by approximately 5.2% and 4.4%, respectively, while the mean power consumption is reduced by approximately 29.5% and 30.7%.

5.3.2. Scenario 2

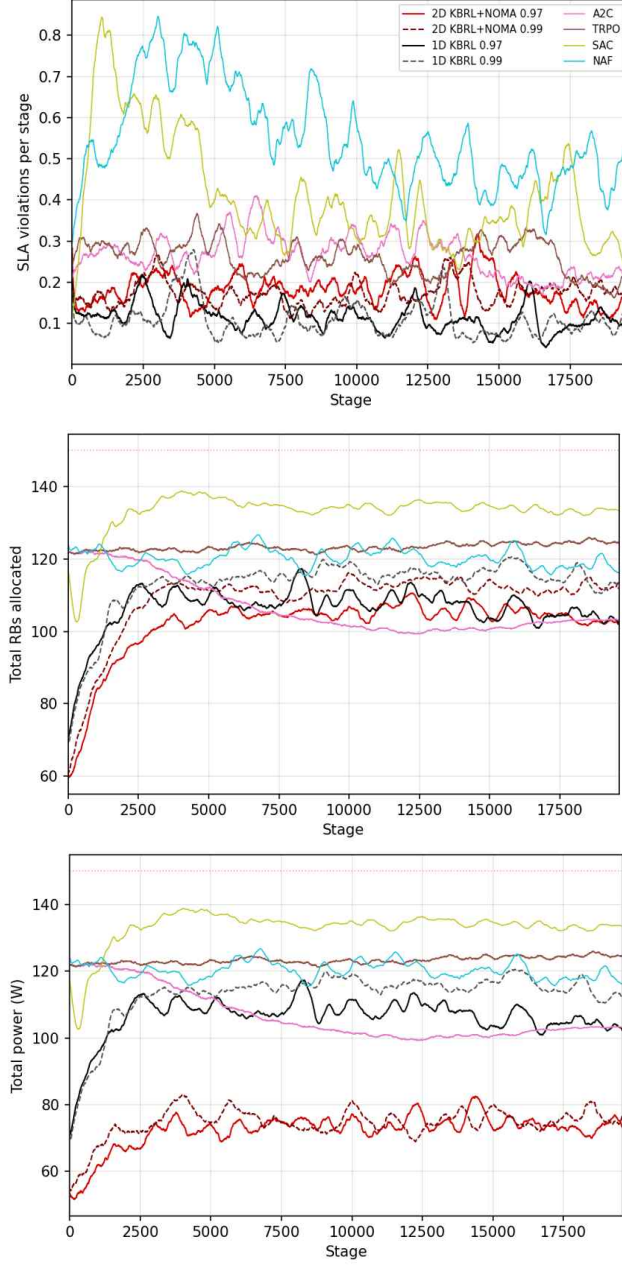


Figure 6: Results for scenario 2.

Table 13: Results for scenario 2: SLA violations, total RBs, and total power allocated.

Algorithm	Viol/stage	Cum. viol.	Mean RB	Mean Power (W)
KBRL $\bar{\delta}=0.97$ — 1D	0.1230	4 922	106.95	106.95
KBRL $\bar{\delta}=0.99$ — 1D	0.1025	4 100	112.19	112.19
KBRL+NOMA $\bar{\delta}=0.97$ — 2D	0.1751	7 004	102.76	73.13
KBRL+NOMA $\bar{\delta}=0.99$ — 2D	0.1614	6 455	109.90	74.70
A2C	0.3121	12 485	103.88	103.88
TRPO	0.2141	8 565	124.19	124.19
SAC	0.3042	12 168	133.49	133.49
NAF	0.4881	18 398	119.65	119.65

Figure 6 and Table 13 show that 2D controller wins decisively on transmit power (73 W vs ≥ 100 W for every other algorithm — a 31 % reduction) and on RB count, but appears worse on the raw violations/stage counter than KBRL–1D. This is because 2D simulator augments [1]'s mMTC layer with a physical channel model — per-repetition SNR, QPSK demodulation threshold, NOMA pairing, and SIC failure cascade — that the 1D simulator omits. Failed decoding keeps sensors in the queue across multiple retry slots, accumulating real wait time in the per-stage delay accumulator, an exhausted sensor (all retransmissions failed) further injects a flat 1500–TTI penalty designed to keep the average-based SLA Boolean responsive to packet drops. Neither contribution is reachable in the 1D simulator, where every scheduled repetition is counted as delivered. The raw mMTC violation excess in 2D therefore reflects the price of physical realism rather than a control regression.

5.3.3. Scenario 3

Table 14: Results for scenario 3: SLA violations, total RBs, and total power allocated.

Algorithm	Viol/stage	Cum. viol.	Mean RB	Mean Power (W)
KBRL $\bar{\delta}=0.97$ — 1D	0.0576	2 306	62.68	62.68
KBRL $\bar{\delta}=0.99$ — 1D	0.0455	1 820	67.45	67.45
KBRL+NOMA $\bar{\delta}=0.97$ — 2D	0.1011	4 046	60.17	42.48
KBRL+NOMA $\bar{\delta}=0.99$ — 2D	0.1834	7 335	65.97	47.50
A2C	0.4971	19 883	62.04	62.04
SAC	0.2545	10 181	87.55	87.55
TD3	0.4048	16 193	64.21	64.21

NAF	1.0374	41 314	77.30	77.30
-----	--------	--------	-------	-------

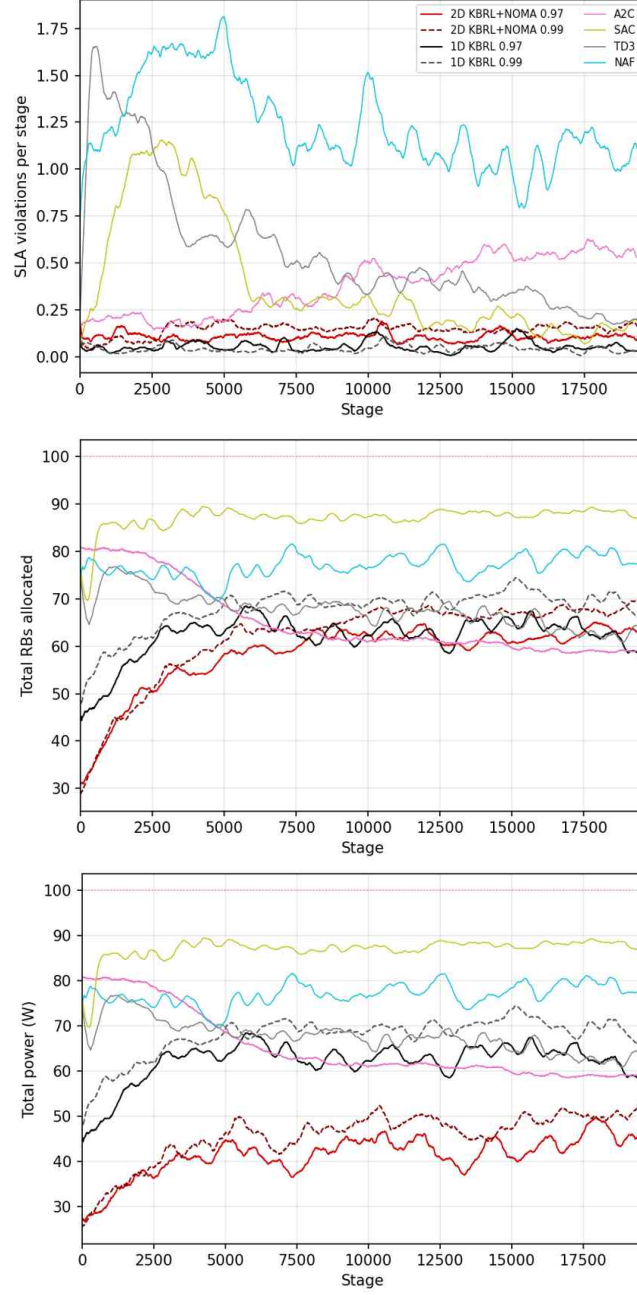


Figure 7: Results for scenario 3.

Table 14 shows that KBRL-1D with $\bar{\delta}=0.99$ has the lowest raw violation count in this scenario (1 820 cumulative). 2D + NOMA, by contrast, is the only family that holds transmit power below 50 W, a 30 % reduction and less RBs use over

the best 1D KBRL. The same simulator–asymmetry as in scenario 2 applies here: 2D's augments [1]'s mMTC layer with a physical channel model, and the violation is the price of physical realism, not a control regression.

5.3.4. Adjustments

The optimiser is not constrained to respect the global budget C , when its proposed allocation exceeds C , the controller falls back to the adjustment

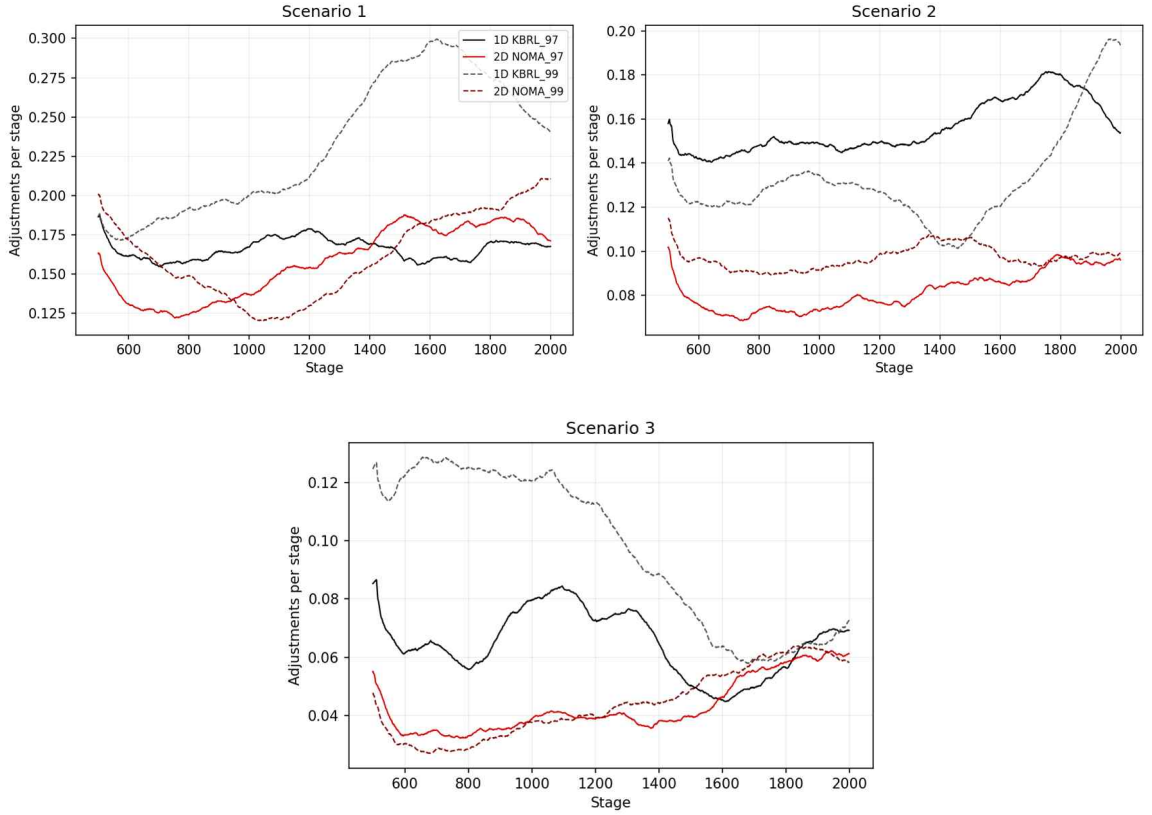


Figure 8: Average number of adjustments per stage in each scenario.

In all three scenarios, the adjustment rate of KBRL–2D stays below 1D KBRL on average: 0.16 vs 0.24 (S1), 0.12 vs 0.17 (S2), and 0.08 vs 0.09 (S3). The 2D joint (RB, power) optimisation lands inside the per–scenario budget more often than 1D's RB–only allocation (Figure 8), requiring less trimming by the overflow projector.

5.3.5. Online classifier accuracy

The second KBRL-specific metric is the empirical accuracy of the per-axis online classifiers

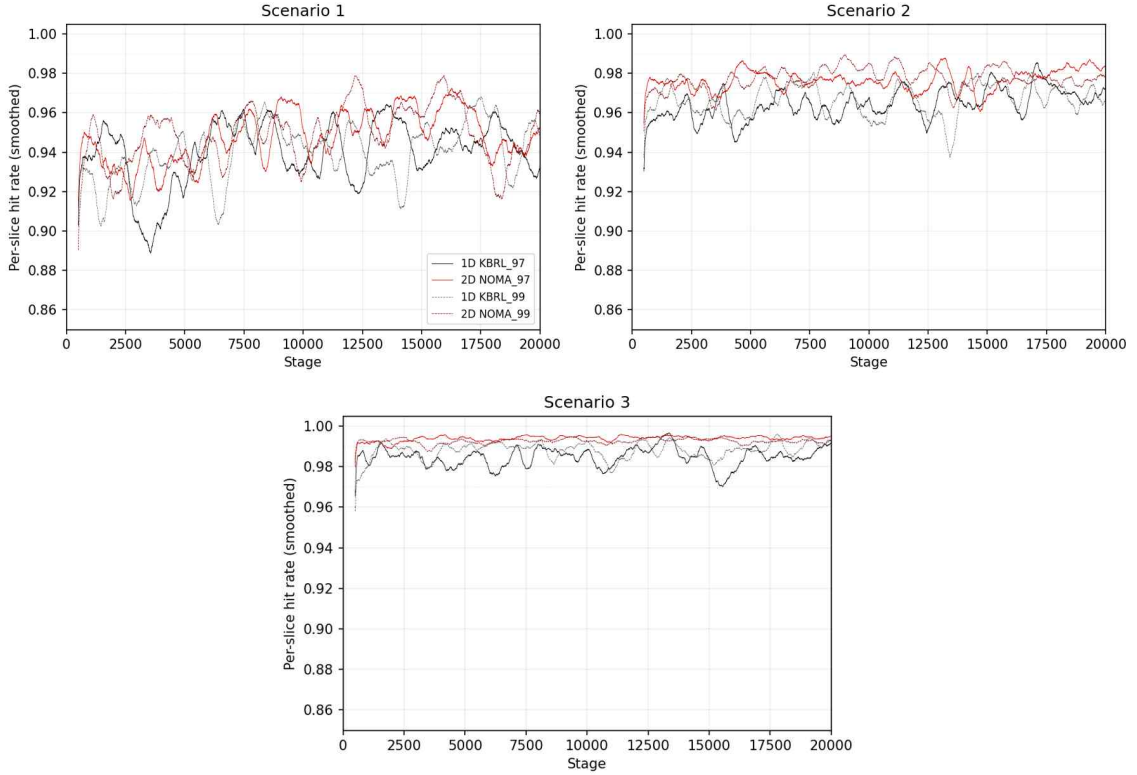


Figure 9: Empirical prediction accuracy of the KBRL classifiers.

In all three scenarios, the classifier hit rate of KBRL-2D stays above 1D KBRL (Figure 9) on average: 0.955 vs 0.943 (S1), 0.979 vs 0.967 (S2), and 0.993 vs 0.986 (S3). The richer 2D action space lets the H-learner separate safe from unsafe allocations more reliably than 1D's RB-only space, and accuracy rises with $\bar{\delta}$ in both controllers, a higher reliability factor places the operating point further inside the feasible region, making the binary classification target easier to predict.

5.4. Limitations and future Directions

Decision-stage complexity: The 2D extension and the intra-slice NOMA pipeline both introduce additional computational cost compared to the original 1D KBRL baseline. the E-learner update increases to $O(2KC)$ due to the joint PRB and per-PRB-power dimensions, while the controller's decoupled binary search requires $O(K(\log C_{\text{PRB}} + \log C_{\text{pr}}))$ instead of the 1D complexity $O(K(\log C_{\text{PRB}}))$. In addition, the NOMA module introduces a DQN forward/backward pass per user pair per TTI with complexity $O(n_{\text{layers}} \cdot n_{\text{neurons}}^3)$, which is absent in the original 1D design. The most noticeable increase comes from the Projectron support-set size $|X_n|$, which reached approximately 1700 per eMBB slice after 40,000 learning stages—around one order of magnitude larger than the value reported in [1] for the 1D controller. This growth reflects the larger number of distinct query points generated by the 2D action space and suggests that periodic support-vector pruning may become necessary for very long training episodes.

Channel model fidelity for mMTC: The two service classes are modelled asymmetrically. The eMBB layer combines 3GPP TS 36.942 large-scale path loss with small-scale fading from EPA/ETU/EVA traces (TR 36.104), giving each UE a realistic frequency-selective multipath profile. The mMTC layer uses distance-dependent path loss and log-normal shadowing only, with no time-varying or frequency-selective fading. The asymmetry tracks the differing sensitivity of each class: eMBB throughput and HARQ depend on multipath across the UE bandwidth, whereas mMTC devices are stationary, narrowband (180 kHz), and evaluated against coverage-class MCL thresholds for which a flat-fading approximation is standard. This is consistent with TR 45.820 Annex D, where small-scale fading is absorbed into receiver-sensitivity values rather than realised per slot. Slice-level KPIs (admission, throughput, latency) are robust to this abstraction; link-reliability KPIs requiring per-slot fading are out of scope and listed as future work.

Service-class scope: Only eMBB and mMTC are evaluated. URLLC, whose SLA semantics differ fundamentally (per-packet reliability and one-way latency rather than averaged throughput or delay), is not considered: the H-learner's binary safe-vs-unsafe classifier and the per-slice E-learner accuracy targets would

need reformulation to handle URLLC's tail-probability contracts.

6. Conclusion

This thesis shows that extending the 1D KBRL framework to jointly manage PRBs and per-RB transmit power provides clear gains in both power efficiency and resource utilization. The proposed 2D KBRL + NOMA controller reduces average transmit power by 30–32% and PRB occupation by 4–5% across all evaluated scenarios, while still preserving the SLA-violation guarantees of the original 1D framework.

These improvements, however, come with a trade-off. Reducing per-RB transmit power pushes some mMTC devices close to the decoding threshold, causing a portion of sensors to exhaust their retry budget before successful decoding. Unlike the original 1D simulator, which cannot model such failures, the proposed framework captures this link-level behavior explicitly, making the energy-efficiency versus reliability trade-off observable. The results therefore confirm that the additional power-control dimension is beneficial for online-safe inter-slice management, but its operating point must still be carefully tuned according to the desired balance between power savings and decoding reliability. Overall, the findings support the use of joint PRB and power allocation combined with NOMA as an effective extension of the original KBRL framework.

Reference

- [1] Juan J. Alcaraz, Fernando Losilla, Andrea Zanella, and Michele Zorzi. Model-based reinforcement learning with kernels for resource allocation in RAN slices. *IEEE Transactions on Wireless Communications*, 22(1):486–501, January 2023.
- [2] 3GPP. System architecture for the 5G System (5GS); Stage 2. Technical Specification TS 23.501, Version 17.5.0, 3rd Generation Partnership Project, June 2022.
- [3] NGMN Alliance. NGMN 5G white paper. Technical report, Next Generation Mobile Networks Alliance, February 2015.
- [4] Xenofon Foukas, Georgios Patounas, Ahmed Elmokashfi, and Mahesh K. Marina. Network slicing in 5G: Survey and challenges. *IEEE Communications Magazine*, 55(5):94–100, May 2017.
- [5] Ibrahim Afolabi, Tarik Taleb, Konstantinos Samdanis, Adlen Ksentini, and Hannu Flinck. Network slicing and softwarization: A survey on principles, enabling technologies, and solutions. *IEEE Communications Surveys & Tutorials*, 20(3):2429–2453, 2018.
- [6] Latif U. Khan, Ibrar Yaqoob, Nguyen H. Tran, Zhu Han, and Choong Seon Hong. Network slicing: Recent advances, taxonomy, requirements, and open research challenges. *IEEE Access*, 8:36009–36028, 2020.
- [7] He Zhang and Vincent W. S. Wong. A two-timescale approach for network slicing in C-RAN. *IEEE Transactions on Vehicular Technology*, 69(6):6656–6669, June 2020.
- [8] Hao Zhou, Medhat Elsayed, and Melike Erol-Kantarci. RAN resource slicing in 5G using multi-agent correlated Q-learning. In *Proceedings of the IEEE 32nd Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, pages 1179–1184. IEEE, September 2021.
- [9] Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT Press, Cambridge, MA, second edition, 2018.
- [10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness,

- Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fiedjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [11] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.
- [12] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37, pages 1889–1897, 2015.
- [13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [14] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48, pages 1928–1937, 2016.
- [15] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016.
- [16] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 1582–1591, 2018.
- [17] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 1861–1870, 2018.
- [18] Shixiang Gu, Timothy Lillicrap, Ilya Sutskever, and Sergey Levine. Continuous deep Q-learning with model-based acceleration. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48, pages 2829–2838, 2016.

- [19] Francesco Orabona, Joseph Keshet, and Barbara Caputo. Bounded kernel-based online learning. *Journal of Machine Learning Research*, 10:2643–2666, 2009.
- [20] Yuya Saito, Yoshihisa Kishiyama, Anass Benjebbour, Takehiro Nakamura, Anxin Li, and Kenichi Higuchi. Non-orthogonal multiple access (NOMA) for cellular future radio access. In *Proceedings of the IEEE 77th Vehicular Technology Conference (VTC Spring)*, pages 1–5. IEEE, 2013.
- [21] Zhiguo Ding, Yuanwei Liu, Jinho Choi, Qi Sun, Maged ElKashlan, Chih-Lin I, and H. Vincent Poor. Application of non-orthogonal multiple access in LTE and 5G networks. *IEEE Communications Magazine*, 55(2):185–191, 2017.
- [22] Thomas M. Cover. Broadcasting. *IEEE Transactions on Information Theory*, 18(1):2–14, 1972.
- [23] Chaofan He, Yang Hu, Yan Chen, and Bing Zeng. Joint power allocation and channel assignment for NOMA with deep reinforcement learning. *IEEE Journal on Selected Areas in Communications*, 37(10):2200–2210, 2019.
- [24] Ayman Rabee and Imad Barhumi. Toward energy-efficient dynamic resource allocation in uplink NOMA systems: Deep reinforcement learning for single and multi-cell NOMA systems. *IEEE Transactions on Vehicular Technology*, 74(6):9313–9327, 2025.
- [25] Asmaa Amer, Sahar Hoteit, and Jalel Ben Othman. Resource allocation for enabled-network-slicing in cooperative NOMA-based systems with underlay D2D communications. In *Proceedings of the IEEE International Conference on Communications (ICC)*, pages 1344–1349. IEEE, 2023.
- [26] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.
- [27] Andrew G. Barto and Sridhar Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, 13(1-2):41–77, 2003.
- [28] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 1726–1734, 2017.
- [29] Alexander S. Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max

- Jaderberg, David Silver, and Koray Kavukcuoglu. FeUdal networks for hierarchical reinforcement learning. In Proceedings of the 34th International Conference on Machine Learning (ICML), volume 70, pages 3540–3549, 2017.
- [30] Sidra Tul Muntaha, Maryam Hafeez, Qasim Z. Ahmed, Faheem A. Khan, Zaharias D. Zaharis, and Pavlos I. Lazaridis. RAN slicing with joint resource allocation for a multi-tenant–multi-service system. *IEEE Transactions on Cognitive Communications and Networking*, 11(3):1927–1939, 2025.
- [31] Duc-Dung Tran, Vu Nguyen Ha, Shree Krishna Sharma, Ti Ti Nguyen, Symeon Chatzinotas, and Petar Popovski. Energy-efficient NOMA for 5G heterogeneous services: A joint optimization and deep reinforcement learning approach. *IEEE Transactions on Communications*, 73(4):2448–2465, 2025.
- [32] Feng Ye, Jiamin Li, Pengcheng Zhu, Dongming Wang, and Xiaohu You. Intelligent hierarchical NOMA-based network slicing in cell-free RAN for 6G systems. *IEEE Transactions on Wireless Communications*, 23(5):4724–4737, 2024.
- [33] 3GPP. Evolved Universal Terrestrial Radio Access (E-UTRA); Radio Frequency (RF) System Scenarios. Technical Specification TS 36.942, Version 15.0.0, 3rd Generation Partnership Project, 2018.
- [34] 3GPP. Evolved Universal Terrestrial Radio Access (E-UTRA); Base Station (BS) Radio Transmission and Reception. Technical Specification TS 36.104, Version 15.15.0, 3rd Generation Partnership Project, 2022.
- [35] Shubham Bhatia, Andreas Mehlführer, Adnan Asadi, et al. A systematic analysis of Narrowband IoT quality of service. *Sensors*, 20(6):1636, 2020.
- [36] 3GPP. Cellular system support for ultra-low complexity and low throughput Internet of Things (CIoT). Technical Report TR 45.820, Version 13.1.0, 3rd Generation Partnership Project, 2015.
- [37] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

5G 네트워크 슬라이싱에서의 다단계 자원 관리: 스펙트럼, 전력 및 NOMA

에야 아로우리

부산대학교 대학원 정보융학공학과

요약

무선 접속 네트워크(RAN) 슬라이싱에서는 각 슬라이스가 서비스 수준 협약(SLA)을 만족할 수 있도록 슬라이스 컨트롤러가 물리 자원 블록(PRB)과 송신 전력을 지속적으로 분배해야 한다. 최근 제안된 커널 기반 강화학습(KBRL) 컨트롤러는 이러한 문제에서 초기 학습 단계의 안정성을 보장하며 기존 딥 강화학습 기반 방법들보다 우수한 성능을 보였지만, 1차원 행동 공간(PRB 개수만 고려)에서만 동작하고 물리 계층 모델이 없는 mMTC 채널을 가정한다. 본 논문에서는 KBRL을 두 가지 측면에서 확장하였다. 첫째, 행동 공간을 PRB 개수와 PRB당 전력을 함께 고려하는 2차원 공간으로 확장하였다. 둘째, eMBB 및 mMTC 슬라이스 모두에 대해 슬라이스 내부 비직교 다중 접속(NOMA) 제어기를 추가하였다. 사용자 장비(UE)는 SINR 차이를 기준으로 페어링되며, 심층 Q-네트워크(DQN)가 successive interference cancellation(SIC) 환경에서 각 페어의 전력 분배 계수를 선택한다. 또한 기존 시뮬레이터의 물리 계층이 없는 mMTC 모델 대신, QPSK 복조 임계값과 SIC 실패 전파를 반영한 3GPP 기반 mMTC 채널 모델을 적용하였다. 총 3개의 시나리오와 30개의 랜덤 시드를 사용한 실험 결과, 제안한 2D KBRL + NOMA 컨트롤러는 동일한 부하 조건에서 기존 1D KBRL 대비 평균 송신 전력을 30~32%, PRB 사용량을 4~5% 감소시키면서도 각 슬라이스의 SLA 요구사항을 유지하였다.